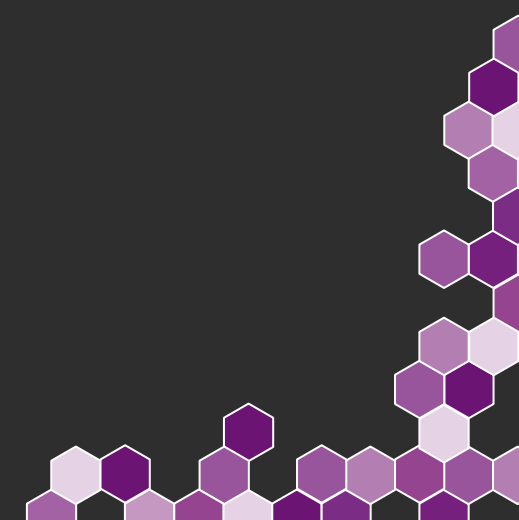




Programmazione in C#



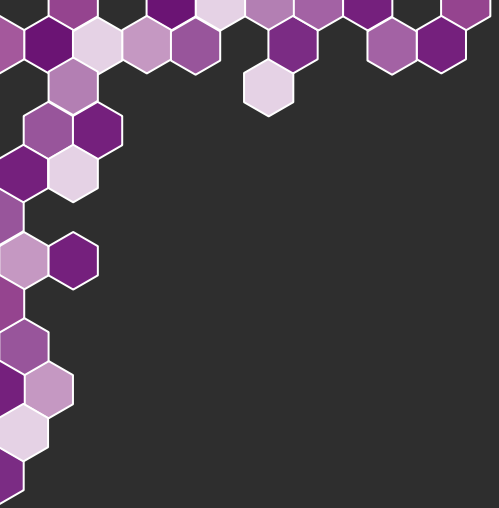


Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni

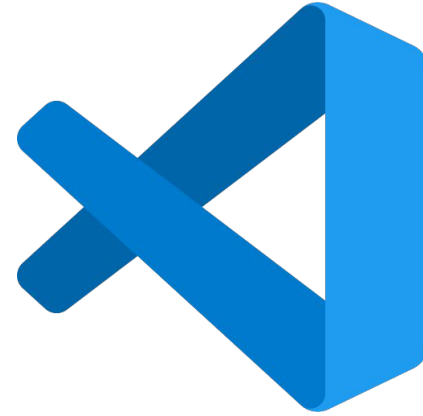


Visual Studio l'IDE della microsoft

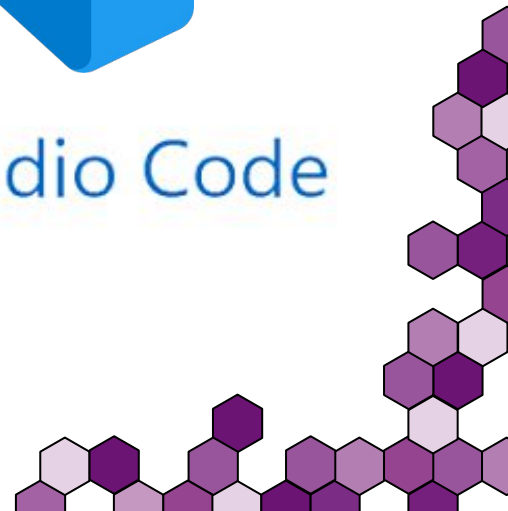
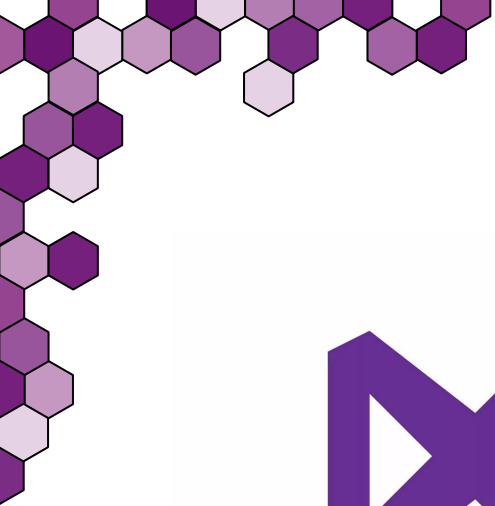




Visual Studio



Visual Studio Code



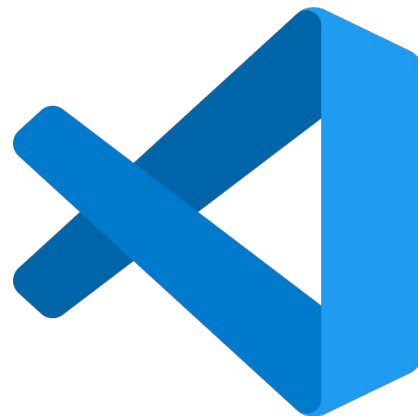


Visual Studio:

- **IDE Completo:** Include strumenti avanzati per lo sviluppo, il debugging e la gestione di progetti complessi.
- **Scalabile:** Ideale per applicazioni enterprise e progetti su larga scala.
- **Supporto Linguaggi:** Supporta solo C#, VB.NET più altri linguaggi ma con poco supporto
- **Strumenti Integrati:** Offre strumenti integrati per testing, profiling, gestione di database e DevOps.
- **Ambiente Pesante:** Richiede più risorse di sistema e può essere più lento su hardware meno potente.

Visual Studio Code:

- **Editor Leggero:** Leggero e veloce, progettato principalmente per l'editing di codice.
- **Flessibilità:** Estremamente estensibile grazie alle numerose estensioni disponibili.
- **Multi-linguaggio:** Supporta molti linguaggi di programmazione tramite estensioni.
- **Personalizzazione:** Altamente personalizzabile con temi, estensioni e impostazioni.
- **Ambiente Snello:** Ideale per progetti leggeri e rapidi sviluppi, utilizza meno risorse di sistema



Visual Studio Code



Visual Studio | 🪟

L'IDE più completo per sviluppatori .NET e C++ in Windows. Completamente pieno di una vasta gamma di strumenti e funzionalità per elevare e migliorare ogni fase dello sviluppo software.

Altre informazioni →

Scarica Visual Studio ▾

Visual Studio Code | 🪟 🍏 🐧

Editor di codice sorgente autonomo eseguito in Windows, macOS e Linux. La scelta migliore per gli sviluppatori JavaScript e Web, con estensioni per supportare praticamente qualsiasi linguaggio di programmazione.

Scopri di più →

Se usi Visual Studio Code, accetti la [licenza](#) & [informativa sulla privacy](#)

Scarica Visual Studio Code ▾

🔔 Ti stai chiedendo quale sia lo strumento più adatto a te? Possiamo aiutarti.

Visual Studio 2022

L'IDE di Visual Studio è una piattaforma di avvio creativa che è possibile usare per progettare, sviluppare, debug e creare codice e quindi pubblicare un'app. Oltre all'editor e al debugger standard, Visual Studio include strumenti per gli IPI, Visual Studio include compilatori, strumenti di completamento del codice, utilità di testing e grafica e molte altre funzionalità per migliorare il processo di sviluppo.

Download ▾

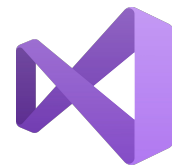
- Community 2022
- Professional 2022
- Enterprise 2022

Versione Gratuita: La Community Edition di Visual Studio 2022 è completamente gratuita per singoli sviluppatori, studenti, e piccole squadre.

Scaricamento:

- Visita il sito ufficiale di [Visual Studio](https://visualstudio.microsoft.com).
- Seleziona "Download" e scegli "Visual Studio 2022 Community".
- Segui le istruzioni per scaricare e installare il software di questa presentazione.

Le versioni di Visual Studio



Visual Studio 2022 Community:

- **Gratuita:** Ideale per singoli sviluppatori, studenti, progetti open source e piccole squadre. Non richiede alcun costo di licenza.
- **Funzionalità Complete:** Offre molte delle funzionalità avanzate presenti nelle versioni Professional ed Enterprise, inclusi strumenti per il debugging, il testing e lo sviluppo di applicazioni web e desktop.
- **Accessibile:** Perfetta per chi è alle prime armi o per chi vuole esplorare il mondo della programmazione senza costi iniziali.

Visual Studio 2022 Professional:

- **Costosa:** Richiede una licenza a pagamento, che può essere un deterrente per studenti e sviluppatori indipendenti.
- **Funzionalità Aggiuntive:** Include alcune funzionalità avanzate per la gestione dei progetti e il lavoro in team, utili per sviluppatori professionisti e team di medie dimensioni.
- **Supporto Tecnico:** Include accesso al supporto tecnico di Microsoft, il che può essere un vantaggio per le aziende.

Visual Studio 2022 Enterprise:

- **Molto Costosa:** La licenza è la più costosa tra le tre, giustificata dalle funzionalità estese e dal supporto completo.
- **Funzionalità Avanzate:** Offre strumenti avanzati per DevOps, test automatizzati, analisi avanzate e gestione su larga scala, ideali per grandi organizzazioni e team di sviluppo complessi.
- **Supporto e Sicurezza:** Include supporto prioritario e strumenti di sicurezza avanzati, fondamentali per grandi imprese.



Visual Studio Installer

Installato Disponibile

Current



Visual Studio Enterprise 2022

17.10.5

Soluzione end-to-end scalabile per team di qualsiasi dimensione

[Termini della licenza](#) | [Note sulla versione](#)

Installa

Altro ▾



Visual Studio Professional 2022

17.10.5

IDE professionale ideale per piccoli team

[Termini della licenza](#) | [Note sulla versione](#)

Installa

Altro ▾



Visual Studio Community 2022

17.10.5

Potente IDE gratuito per studenti, collaboratori open-source e singoli utenti

[Termini della licenza](#) | [Note sulla versione](#)

Installa

Altro ▾

Novità per gli sviluppatori

[Visual Studio tip: Using Ctrl-L to delete a line with and without Clipboard backup](#)

Here is one Visual Studio tip from my colleague G...

mercoledì 24 luglio 2024

[Three New Premium Visual Studio Subscriber Benefits Announced](#)

A Visual Studio Subscription, whether Professiona...

giovedì 18 luglio 2024

[Making Visual Studio a bit more visual](#)

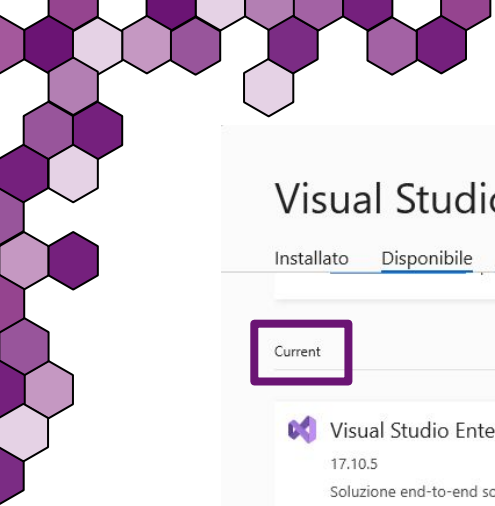
Any web, desktop, or mobile developer works wit...

martedì 16 luglio 2024

[Visualizza altre novità per gli sviluppatori Microso...](#)

Per assistenza, visitare la pagina della [Developer Community Microsoft](#), oppure contattarci tramite la pagina [Supporto tecnico di Visual Studio](#).

Versione 3.11.2167.20057 del programma di installazione





Installazione e pacchetti

Installazione in corso — Visual Studio Community 2022 — 17.10.5

Carichi di lavoro Singoli componenti Language Pack Percorsi di installazione

Sei serve aiuto per la scelta dei componenti da installare? [Altre informazioni](#)

Web e cloud (4)

- ☐ Sviluppo ASP.NET e Web
Consente di creare applicazioni Web con ASP.NET Core, ASP.NET, HTML/JavaScript e contenitori, tra cui il supporto...
- ☐ Sviluppo di Azure
SDK di Azure, strumenti e progetti per sviluppare app cloud e creare risorse usando .NET e .NET Framework In...
- ☐ Sviluppo Python
Modifica, debug, sviluppo interattivo e controllo del codice sorgente per Python.
- ☐ Sviluppo Node.js
Consente di compilare applicazioni di rete scalabili con Node.js, un runtime JavaScript basato su eventi asincroni.

Desktop e dispositivi mobili (5)

- ☐ Sviluppo dell'interfaccia utente di app .NET multipla...
Consente di creare app Android, iOS, Windows e Mac da una singola codebase usando C# con .NET MAUI.
- ☐ Sviluppo per desktop .NET
Consente di creare applicazioni WPF, Windows Form e console con C#, Visual Basic e F# con .NET e .NET Frame...
- ☐ Sviluppo di applicazioni desktop con C++
Consente di creare app C++ moderne per Windows usando gli strumenti preferiti, inclusi MSVC, Clang, CMak...
- ☐ Sviluppo applicazione Windows
Consentono di compilare applicazioni per la piattaforma Windows usando WinUI con C# o facoltativamente C++.
- ☐ Sviluppo di applicazioni per dispositivi mobili con C++
Consente di compilare applicazioni multiplatforma per iOS, Android o Windows con C++.

Giochi (2)

- ☐ Sviluppo di giochi con Unity
Consente di creare giochi 2D e 3D con Unity, un potente ambiente di sviluppo multiplatforma.
- ☐ Sviluppo di giochi con C++
Sfruttare tutte le funzionalità di C++ per compilare giochi professionali basati su DirectX, Unreal o Cocos2d.

Altri set di strumenti (5)

- ☐ Elaborazione ed archiviazione dati
- ☐ Applicazioni analitiche e di analisi scientifica dei dati
- ☐ Sviluppo di estensioni di Visual Studio

Percorso
C:\Program Files\Microsoft Visual Studio\2022\Community [Cambia...](#)

Continuando si accetta la [licenza](#) relativa all'edizione di Visual Studio selezionata. Viene anche offerta la possibilità di scaricare altro software con Visual Studio. Tale software viene concesso in licenza separatamente, come indicato nelle [Comunicazioni di terze parti](#) nella licenza associata. Continuando si accettano anche tali licenze.

Obbligatorio

Facoltativi

1,43 GB

Spazio totale richiesto 1,43 GB

Installa durante il download [Installa](#)



Installazione in corso — Visual Studio Community 2022 — 17.10.5

Carichi di lavoro

Singoli componenti

Language Pack

Percorsi di installazione

È possibile aggiungere altri Language Pack all'installazione di Visual Studio.

- ☐ Ceco
- ☐ Cinese (semplificato)
- ☐ Cinese (tradizionale)
- ☒ Coreano
- ☐ Francese
- ☐ Giapponese
- ☒ Inglese
- ☒ Italiano
- ☐ Polacco
- ☐ Portoghese (Brasile)
- ☐ Russo
- ☐ Spagnolo
- ☐ Tedesco
- ☐ Turco

Potete mettere solo inglese o solo italiano

Installazione completa



Installazione in corso — Visual Studio Community 2022 — 17.10.5

Carichi di lavoro Singoli componenti Language Pack Percorsi di installazione

Se serve aiuto per la scelta dei componenti da installare? [Altre informazioni](#)

Web e cloud (4)

Sviluppo ASP.NET e Web
Consente di creare applicazioni Web con ASP.NET Core, ASP.NET, HTML/JavaScript e contenitori, tra cui il support...

Sviluppo di Azure
SDK di Azure, strumenti e progetti per sviluppare app cloud e creare risorse usando .NET e .NET Framework. In...

Sviluppo Python
Modifica, debug, sviluppo interattivo e controllo del codice sorgente per Python.

Sviluppo Node.js
Consente di compilare applicazioni di rete scalabili con Node.js, un runtime JavaScript basato su eventi asincroni.

Desktop e dispositivi mobili (5)

Sviluppo dell'interfaccia utente di app .NET multipla...
Consente di creare app Android, iOS, Windows e Mac da una singola codebase usando C# con .NET MAUI.

Sviluppo per desktop .NET
Consente di creare applicazioni WPF, Windows Form e console con C#, Visual Basic e F# con .NET e .NET Frame...

Sviluppo di applicazioni desktop con C++
Consente di creare app C++ moderne per Windows usando gli strumenti preferiti, inclusi MSVC, Clang, CMak...

Sviluppo applicazione Windows
Consentono di compilare applicazioni per la piattaforma Windows usando WinUI con C# o facoltativamente C++.

Sviluppo di applicazioni per dispositivi mobili con C++
Consente di compilare applicazioni multiplatforma per iOS, Android o Windows con C++.

Giochi (2)

Sviluppo di giochi con Unity
Consente di creare giochi 2D e 3D con Unity, un potente ambiente di sviluppo multiplatforma.

Sviluppo di giochi con C++
Sfruttare tutte le funzionalità di C++ per compilare giochi professionali basati su DirectX, Unreal o Cocos2d.

Altri set di strumenti (5)

Elaborazione ed archiviazione dati

Applicazioni analitiche e di analisi scientifica dei dati

Sviluppo di estensioni di Visual Studio

Percorso

C:\Program Files\Microsoft Visual Studio\2022\Community [Cambia...](#)

Continuando si accetta la [licenza](#) relativa all'edizione di Visual Studio selezionata. Viene anche offerta la possibilità di scaricare altro software con Visual Studio. Tale software viene concesso in licenza separatamente, come indicato nelle [Comunicazioni di terze parti](#) nella licenza associata. Continuando si accettano anche tali licenze.

Dettagli di installazione

- Editor principale di Visual Studio
- Sviluppo per desktop .NET
- Sviluppo applicazione Windows
- Sviluppo di giochi con Unity
 - ▾ Inclusi
 - ✓ Visual Studio Tools per Unity
 - ✓ C# e Visual Basic
 - ▾ Facoltativi
 - ✓ Hub Unity
 - ✓ IntelliCode
 - ✓ Strumenti HLSL
 - ✓ GitHub Copilot

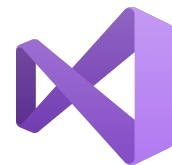
17,19 GB



Ritrovare i componenti non supportati

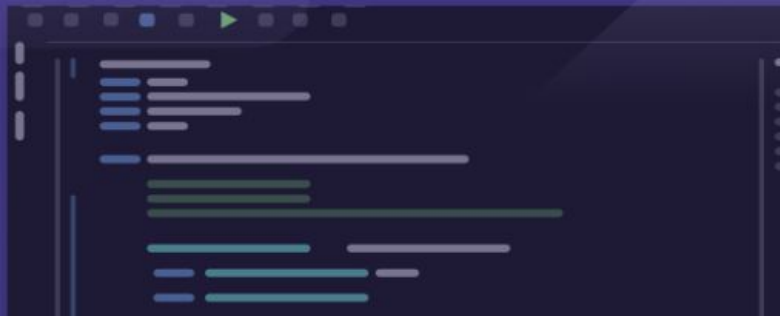
Spazio totale richiesto 17,19 GB

Installa durante il download



Accedi a Visual Studio

Sincronizza le impostazioni tra i dispositivi, collabora in tempo reale e integra senza problemi con i servizi di Azure.



Mail scolastica

Accedi

Crea un account

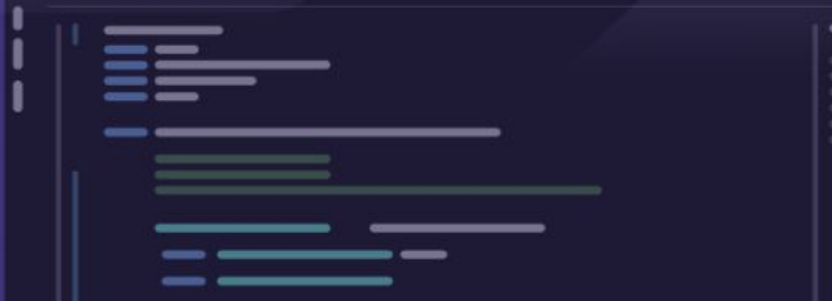
[Ignora per ora.](#)



Personalizzare l'esperienza di Visual Studio

Ottimizzare il layout e le scelte rapide da tastiera per il flusso di lavoro. Selezionare il tema colori più adatto allo stile.

È sempre possibile modificare queste impostazioni in un secondo momento.

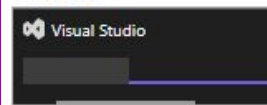


Impostazioni di sviluppo

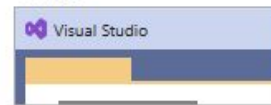
Generale

Scegli tema colori

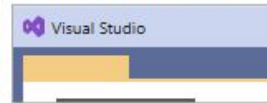
☒ Scuro



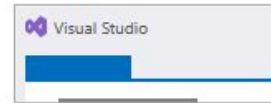
☐ Blu



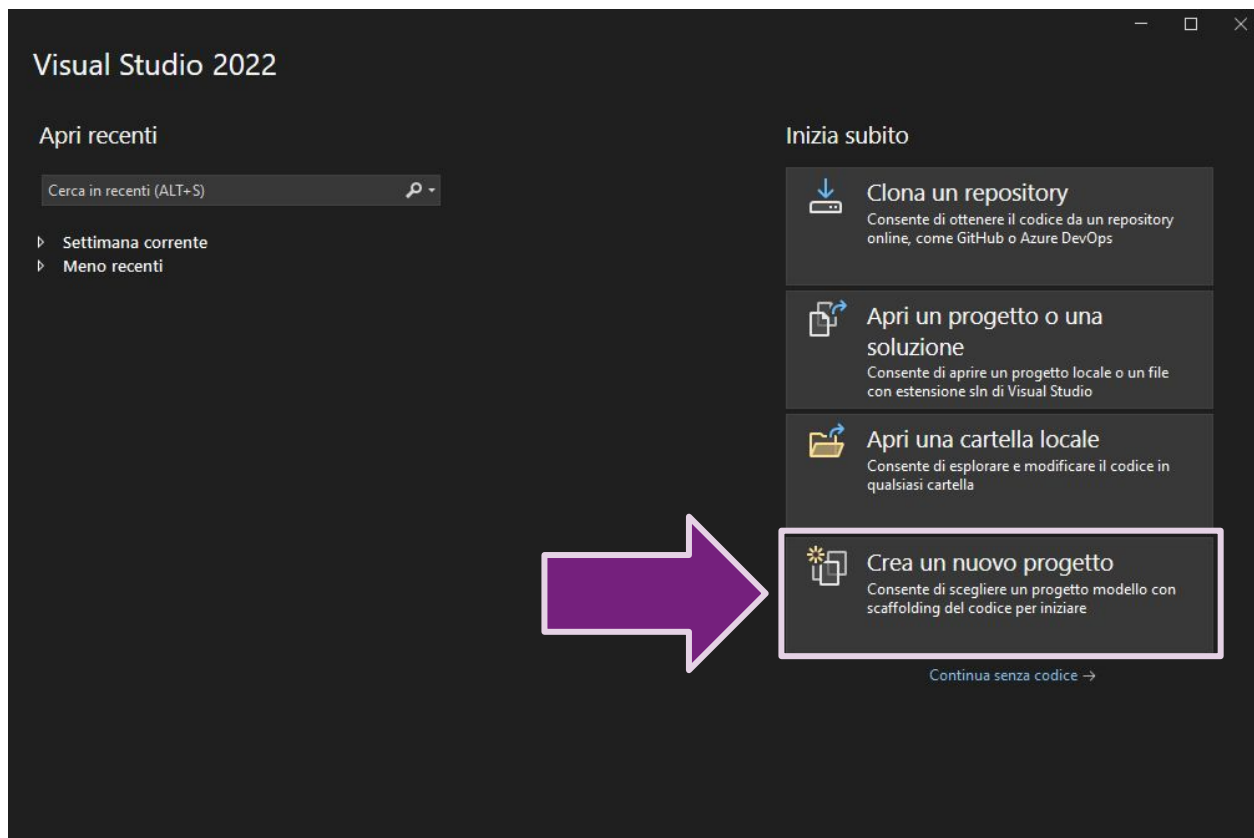
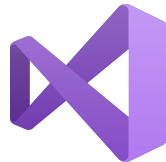
☐ Blu (massimo contrasto)

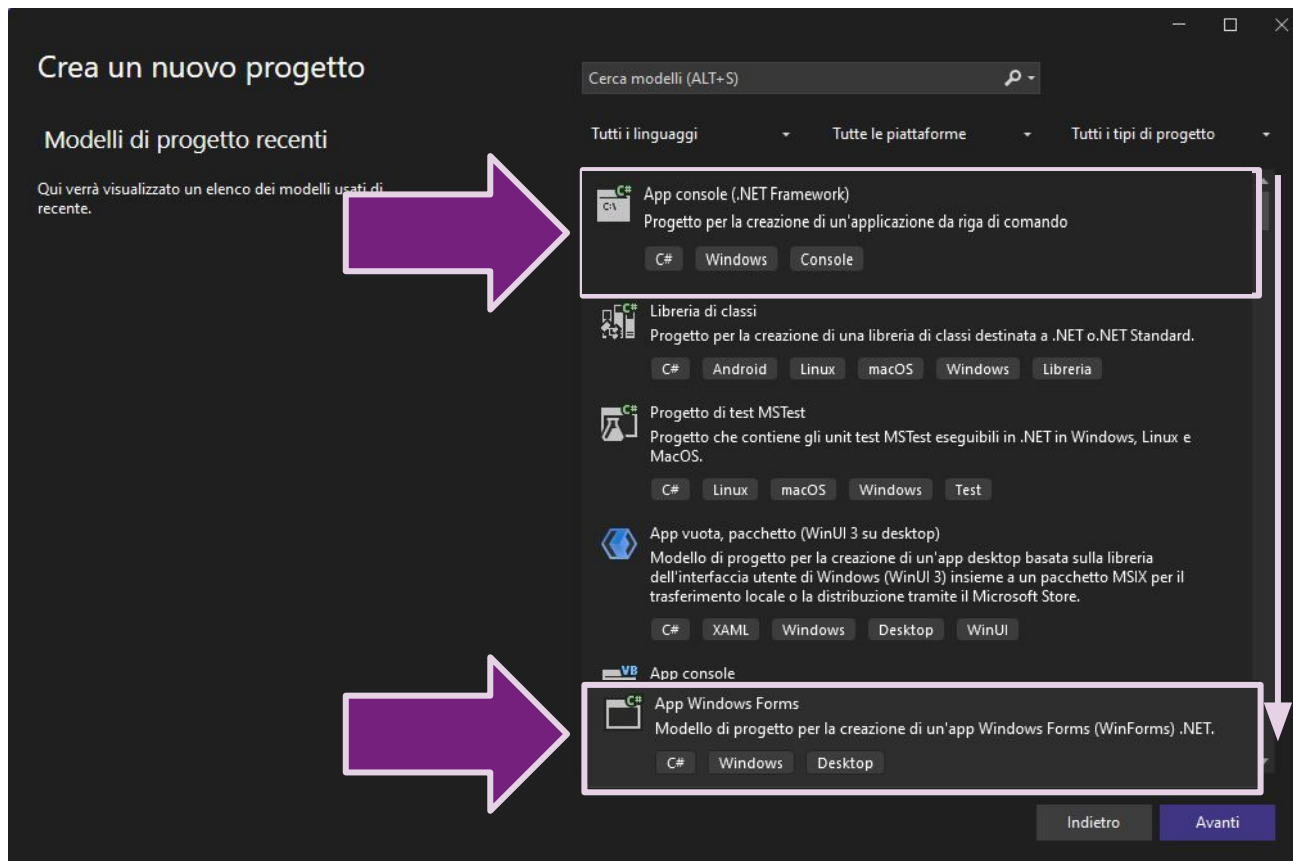


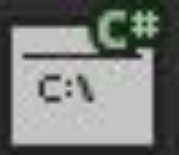
☐ Chiaro



Avvia Visual Studio

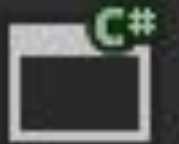






App console (.Net Framework)

- Le applicazioni console utilizzano un'interfaccia a linea di comando (CLI), dove l'interazione con l'utente avviene tramite input e output testuale.
- Ideali per creare programmi semplici, script, e strumenti di utilità che non richiedono una GUI (Graphical User Interface).
- Perfette per i principianti che vogliono imparare le basi della programmazione senza la complessità della gestione della GUI.



App Windows form

- Le applicazioni Windows Forms offrono un'interfaccia grafica con finestre, pulsanti, caselle di testo, e altri controlli interattivi.
- Ideali per creare applicazioni desktop con interfaccia utente ricche e interattive.
- Forniscono una vasta gamma di controlli predefiniti per costruire rapidamente interfacce utente complesse.
- Visual Studio include strumenti di design drag-and-drop per facilitare la creazione e la disposizione di controlli sulla finestra.



Configura il nuovo progetto

App console C# Linux macOS Windows Console

Nome del progetto
ConsoleApp

Percorso
C:\Users\Marco\source\repos

Nome soluzione ⓘ
ConsoleApp

☐ Inserisci soluzione e progetto nella stessa directory

Progetto verrà creato in "C:\Users\Marco\source\repos\ConsoleApp\ConsoleApp"

Indietro Avanti

Nome progetto,
cartella e soluzione

Cambiare percorso
al Desktop

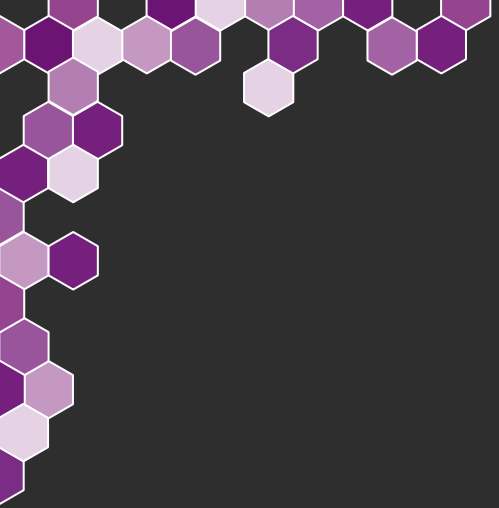


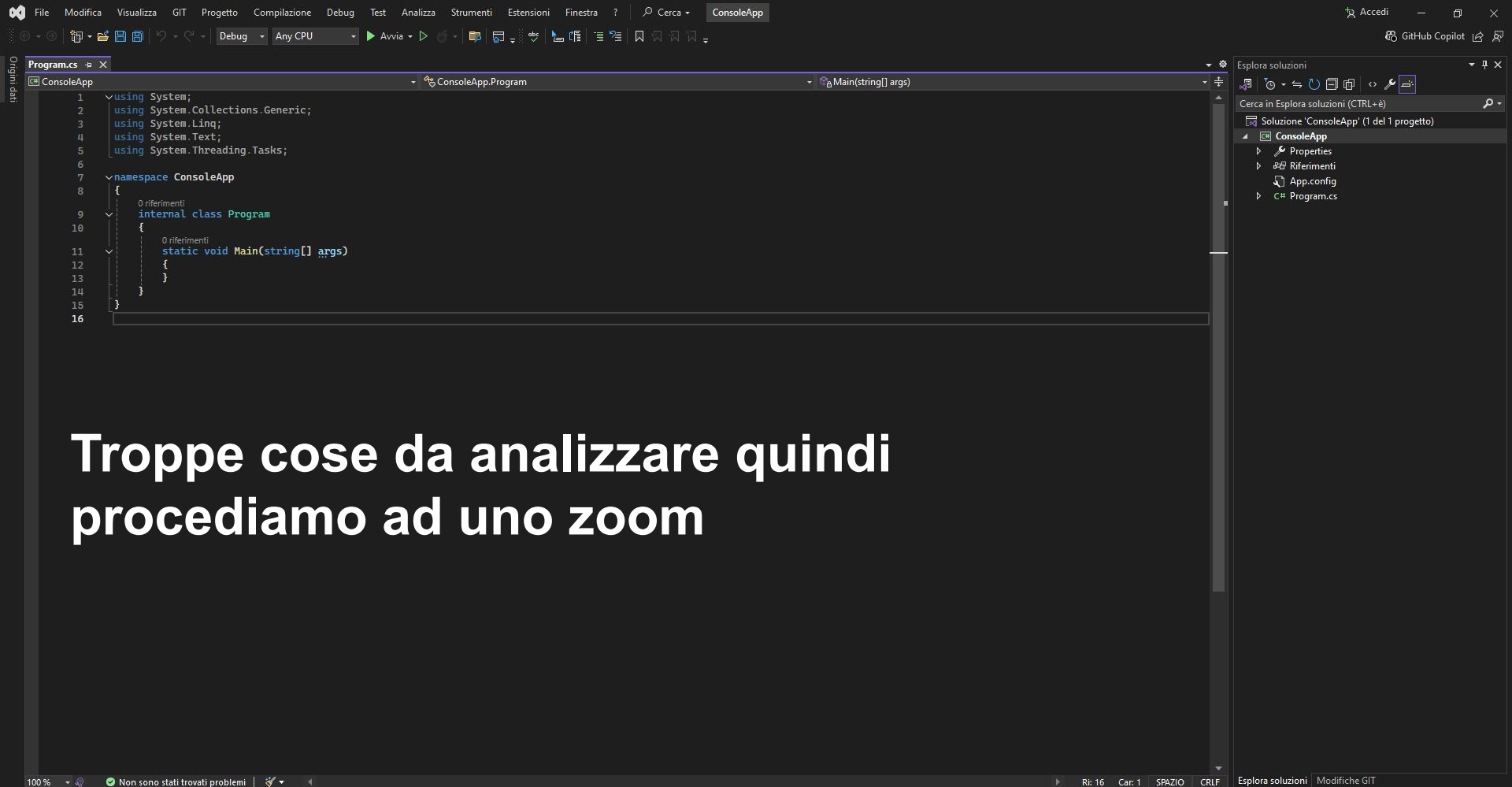
Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



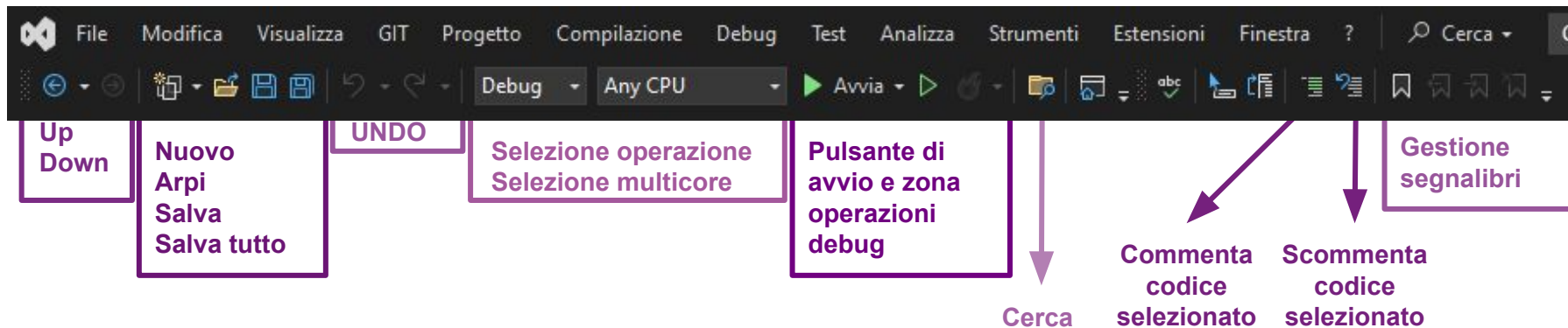
App console presentazione e IO





Troppe cose da analizzare quindi
procediamo ad uno zoom

Prova le varie opzioni e scopri l'IDE

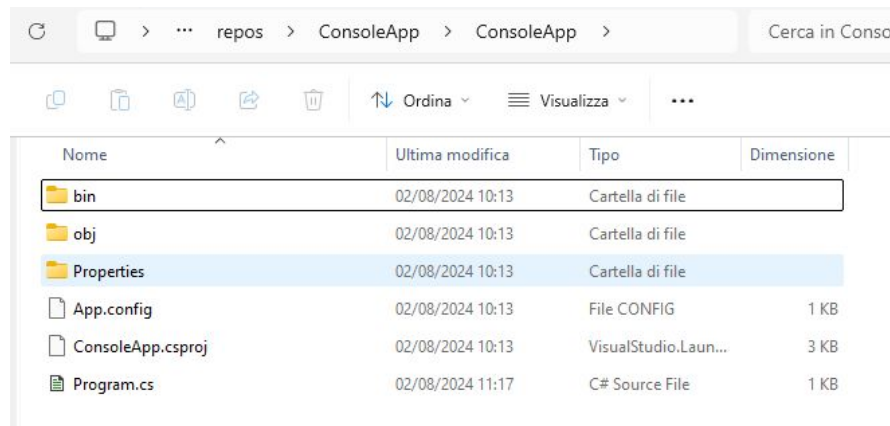
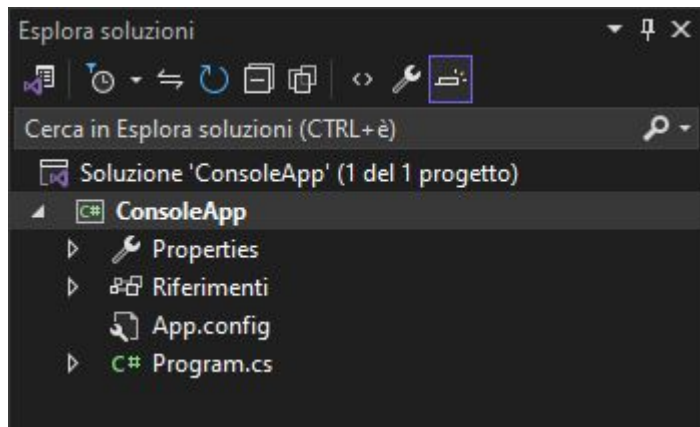


Menù a discesa sulla cima dell'IDE:

Contiene le voci principali del menu, come: "File", "Modifica", "Visualizza", "GIT", "Progetto", "Compilazione", "Debug", "Test", "Analizza", "Strumenti", "Estensioni", "Finestra", e "?".

Questi menù offrono accesso a una vasta gamma di funzionalità dell'IDE, tra cui la gestione dei file, le impostazioni del progetto, gli strumenti di debug e molte altre opzioni. Il professore vi incoraggia a esplorare personalmente e approfonditamente i vari menu e le funzionalità disponibili. È importante ricordare che questo strumento sarà fondamentale per il vostro lavoro nei prossimi anni, e comprendere appieno il suo funzionamento è una parte essenziale della vostra preparazione e valutazione. L'esplorazione e la familiarizzazione con Visual Studio vi permetteranno di sfruttare al meglio tutte le sue potenzialità, migliorando così le vostre competenze e il vostro successo accademico e professionale.

Esplora soluzioni



L'**esplora soluzioni** in Visual Studio è uno strumento fondamentale per la gestione dei progetti di sviluppo, fornendo una rappresentazione visuale e organizzata di tutti i file e le risorse. Ogni elemento visibile in esplora soluzioni ha un corrispettivo fisico nella cartella del progetto nel file system. Ad esempio, file come **Program.cs** o **App.config** corrispondono a file fisici nella directory del progetto. Le modifiche apportate ai file tramite l'IDE vengono salvate direttamente nei file fisici.

Esplora soluzioni rappresenta i file e le cartelle del progetto in una struttura ad albero, che rispecchia fedelmente la disposizione nel file system. Ad esempio, una cartella denominata "Properties" in esplora soluzioni corrisponde a una cartella con lo stesso nome nella directory del progetto. Oltre ai file di codice sorgente, esplora soluzioni gestisce anche le risorse come immagini, file di configurazione e dipendenze esterne.

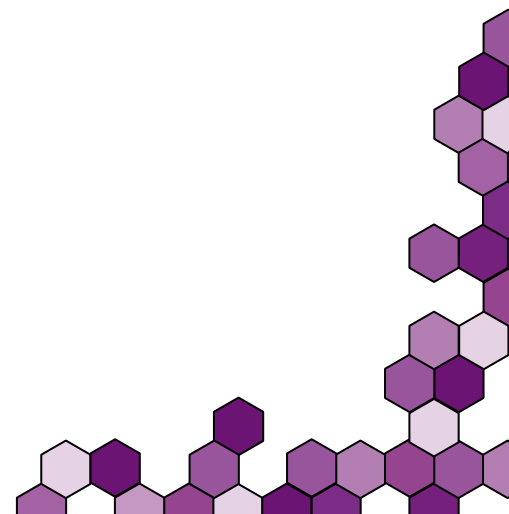
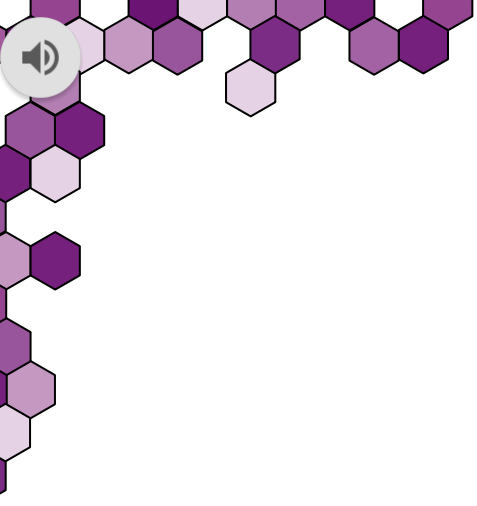
Spiegazione codice di base

```
using System; //contiene classi fondamentali come Console, utilizzata per le operazioni di input/output.
using System.Collections.Generic; //Libreria per la gestione delle collezioni di elementi
using System.Linq; //Libreria per la gestione di query (prevista di base ma non la useremo)
using System.Text; //contiene classi che rappresentano set di caratteri, e manipolazioni di stringhe.
using System.Threading.Tasks; //fornisce tipi per la programmazione asincrona (che useremo quest'anno)

namespace ConsoleApp
//Definisce un namespace chiamato "ConsoleApp". Aiuta a organizzare il codice e a prevenire conflitti di nome.
{
    internal class Program
    // Definisce una classe chiamata "Program". La parola chiave "internal" indica che la classe è accessibile solo all'interno
    //dell'assembly corrente.

    {
        static void Main(string[] args)
        /* Definisce il metodo Main, che è il punto di ingresso dell'applicazione console. La parola chiave "static" indica
        che il metodo appartiene alla classe "Program" piuttosto che a un'istanza della classe. Il tipo di ritorno "void"
        indica che il metodo non restituisce alcun valore. Il parametro "string[] args" rappresenta un array di argomenti
        passati dalla linea di comando.*/
        {
            // Corpo del metodo Main. Attualmente vuoto, il che significa che l'applicazione non esegue alcuna operazione.
        }
    }
}
```





Output

```
namespace ConsoleApp
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Console.WriteLine("Hello, world!");
```

*/*Console: È una classe statica appartenente al namespace System.*

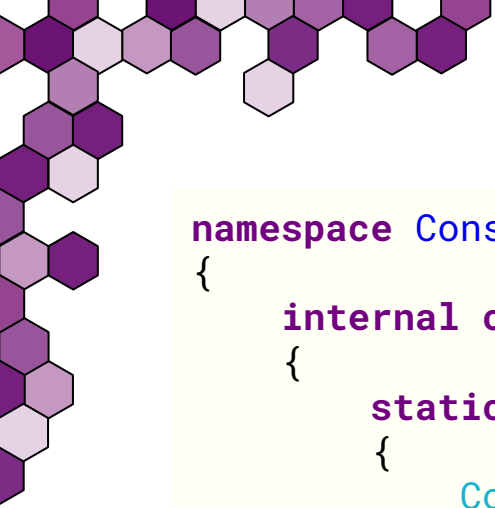
Fornisce metodi per interagire con la console, ossia l'interfaccia a linea di comando.

WriteLine: È un metodo della classe Console. Questo metodo stampa una linea di testo sulla console e aggiunge automaticamente un carattere di nuova riga alla fine, spostando il cursore alla linea successiva./**

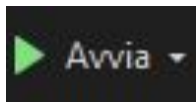
```
        }
```

```
    }
```

```
}
```



Esecuzione



Lo esegue e sparisce anche a voi????????

```
Mostra output di: Debug
'ConsoleApp.exe' (CLR v4.0.30319: DefaultDomain): caricamento di 'C:\WINDOWS\Microsoft.Net\assembly\GAC_32mscorlib\v4.0.4.0.0__b77a5c561934e089\mscorlib.dll' completato. Caricamento dei simboli ignorato. Il
modulo è ottimizzato e l'opzione del debugger 'Solo codice utente' è abilitata.
'ConsoleApp.exe' (CLR v4.0.30319: DefaultDomain): caricamento di 'C:\Users\Marco\source\repos\ConsoleApp\ConsoleApp\bin\Debug\ConsoleApp.exe' completato. Simboli caricati.
Il programma '[12472] ConsoleApp.exe' è terminato con il codice 0 (0x0).
```

Quando si esegue un'applicazione console in C#, la finestra della console si chiude immediatamente dopo aver completato l'esecuzione del codice, senza permettere all'utente di vedere l'output generato. Questo accade perché, dopo aver eseguito l'ultimo comando del metodo **Main**, il programma termina e la finestra della console viene chiusa automaticamente.

Come risolviamo?

Gli diamo un motivo per aspettare

Input

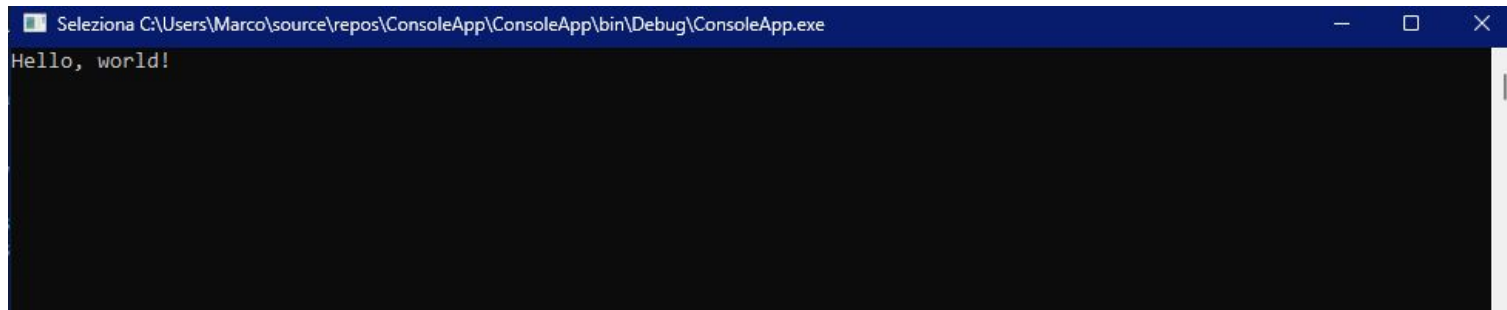
```
namespace ConsoleApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.ReadLine();
            /*
             * ReadLine: È un metodo della classe Console che legge una linea di input
             * dalla console. Questo metodo attende che l'utente inserisca del testo
             * seguito dalla pressione del tasto Invio.
             */
        }
    }
}
```

ReadLine: È un metodo della classe Console che legge una linea di input dalla console. Questo metodo attende che l'utente inserisca del testo seguito dalla pressione del tasto Invio.

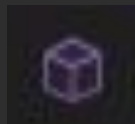
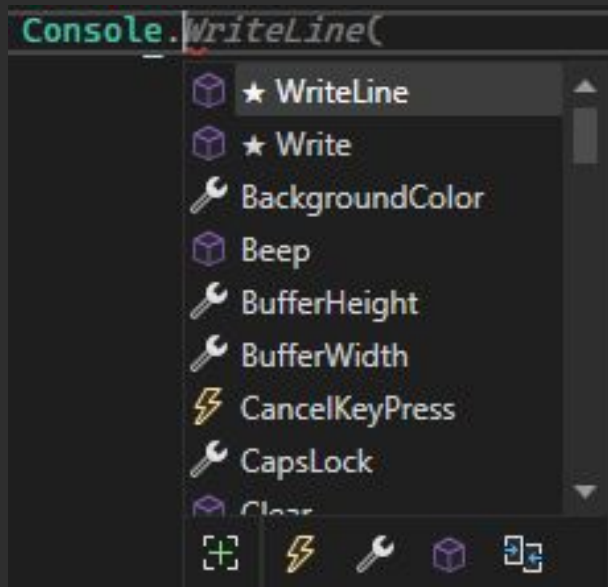
**/*

Primo Hello world

```
namespace ConsoleApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, world!");
            Console.ReadLine();
        }
    }
}
```



La classe Console



Metodi



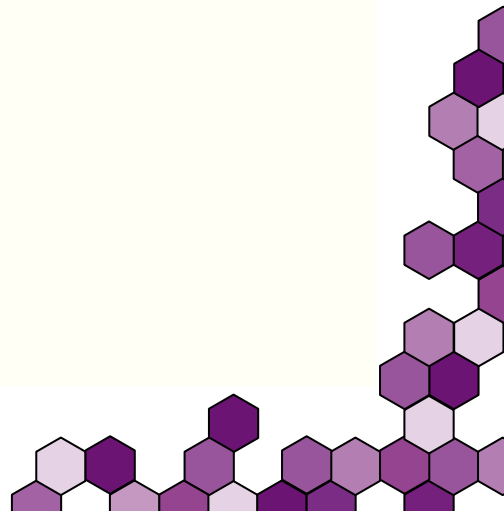
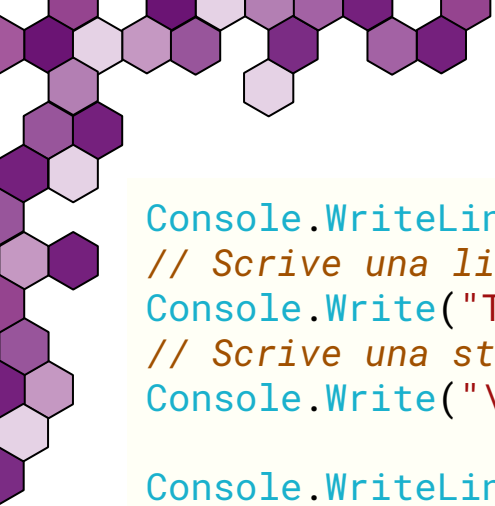
Proprietà



Eventi

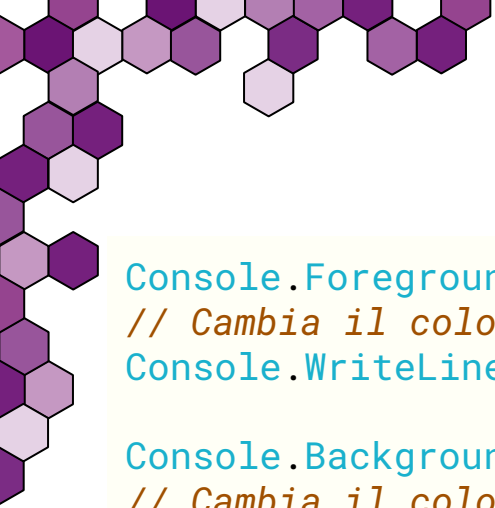
La classe Console

```
Console.WriteLine("Hello, world!");  
// Scrive una linea di testo sulla console con un carattere di nuova riga  
Console.Write("This is on the same line.");  
// Scrive una stringa sulla console senza nuova riga  
Console.Write("\n");  
  
Console.WriteLine("Please enter your name:");  
string name = Console.ReadLine();  
// Legge una linea di input dalla console  
Console.WriteLine($"Hello, {name}!");  
  
Console.WriteLine("Press any key to end...");  
Console.ReadKey();  
// Legge il prossimo tasto premuto dall'utente  
  
Console.Clear(); // Cancella la console
```



La classe Console

```
Console.ForegroundColor = ConsoleColor.Green;  
// Cambia il colore del testo della console  
Console.WriteLine("Text in green!"); Console.ReadLine();  
  
Console.BackgroundColor = ConsoleColor.White;  
// Cambia il colore di sfondo della console  
Console.Clear(); // Deve essere chiamato per applicare il colore di sfondo  
Console.WriteLine("Background color changed!"); Console.ReadLine();  
  
// Ripristina i colori predefiniti  
Console.ResetColor(); Console.WriteLine("Colors reset to defaults!");
```



La classe Console

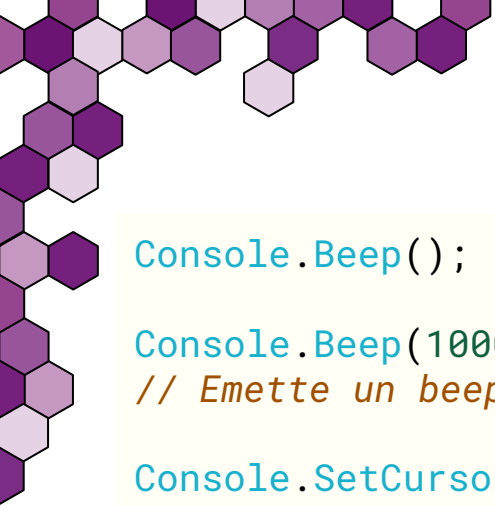
```
Console.Beep(); // Emette un beep
```

```
Console.Beep(1000, 500);  
// Emette un beep con frequenza e durata specificate 1000 Hz per 500 ms
```

```
Console.SetCursorPosition(10, 5); // Imposta la posizione del cursore  
Console.WriteLine("Cursor moved!");
```

```
Console.SetWindowSize(100, 30); // Imposta la dimensione della finestra  
della console
```

```
Console.Title = "Console Methods Demo"; // Imposta il titolo della  
finestra della console
```



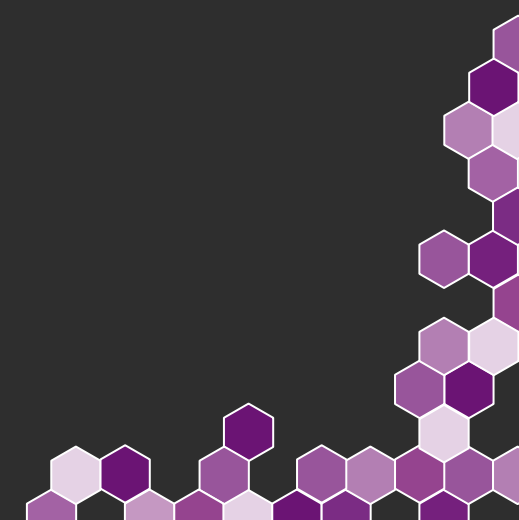
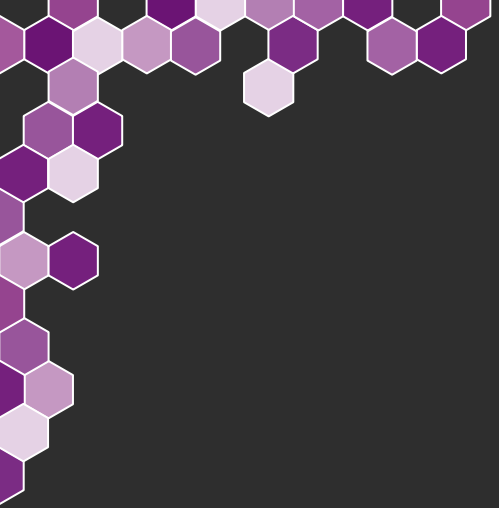


Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



Tipi di variabili



Linguaggio tipizzato

C# è un linguaggio di programmazione **fortemente tipizzato**, il che significa che ogni variabile e costante deve avere un **tipo specifico** dichiarato al momento della definizione. Questo tipo non può cambiare durante l'esecuzione del programma, garantendo una maggiore **sicurezza del codice** e prevenendo errori comuni legati alla gestione dei dati. Ad esempio, se si dichiara una variabile come `int`, essa potrà contenere solo **numeri interi** e non potrà essere assegnato un valore di tipo diverso, come una stringa o un decimale. Questo sistema di **tipizzazione rigorosa** permette al compilatore di rilevare molteplici errori già durante la **fase di compilazione**, piuttosto che durante l'esecuzione del programma, contribuendo a creare software più **affidabile e stabile**. Inoltre, la **tipizzazione statica** di C# facilita l'**ottimizzazione del codice** e offre una **documentazione implicita** del tipo di dati utilizzati, migliorando la **leggibilità** e la **manutenzione** del codice stesso.

I tipi numerici

```
int integerExample = 0; // Tipo intero con segno  
//Dimensione: 4 byte, Min: -2,147,483,648, Max: 2,147,483,647
```

```
float floatExample = 0.0f; //Tipo virgola mobile a precisione singola  
//Dimensione: 4 byte, Min:  $\pm 1.5e-45$ , Max:  $\pm 3.4e38$ , Precisione: ~6-9 cifre
```

```
double doubleExample = 0.0; // Tipo virgola mobile a doppia precisione  
//Dimensione: 8 byte, Min:  $\pm 5.0e-324$ , Max:  $\pm 1.7e308$ , Precisione: ~15-17 cifre
```

```
decimal decimalExample = 0.0m; //Tipo decimale per valori finanziari  
//Dimensione: 16 byte, Min:  $\pm 1.0 \times 10^{-28}$ , Max:  $\pm 7.9 \times 10^{28}$ ,  
//Precisione: 28-29 cifre
```

I tipi numerici speciali

```
uint unsignedIntegerExample = 0; // Tipo intero senza  
//Dimensione: 4 byte, Min: 0, Max: 4,294,967,295
```

```
sbyte signedByteExample = 0; // Tipo intero con segno piccolo  
//Dimensione: 1 byte, Min: -128, Max: 127
```

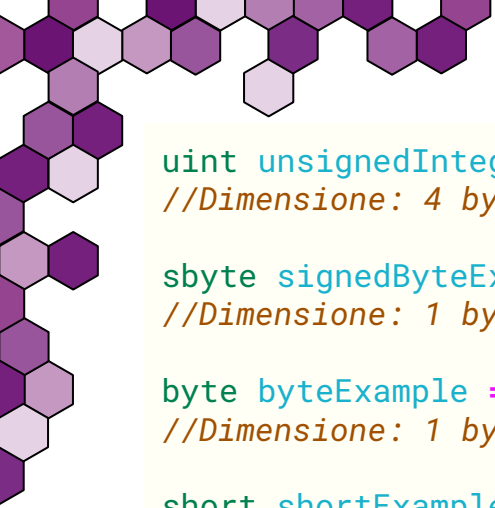
```
byte byteExample = 0; // Tipo intero senza segno piccolo  
//Dimensione: 1 byte, Min: 0, Max: 255
```

```
short shortExample = 0; // Tipo intero con segno (16-bit)  
//Dimensione: 2 byte, Min: -32,768, Max: 32,767
```

```
ushort unsignedShortExample = 0; // Tipo intero senza segno  
//Dimensione: 2 byte, Min: 0, Max: 65,535
```

```
long longExample = 0L; // Tipo intero con segno  
//Dimensione: 8 byte, Min: -9,223,372,036,854,775,808, Max: 9,223,372,036,854,775,807
```

```
ulong unsignedLongExample = 0UL; // Tipo intero senza segno  
//Dimensione: 8 byte, Min: 0, Max: 18,446,744,073,709,551,615
```

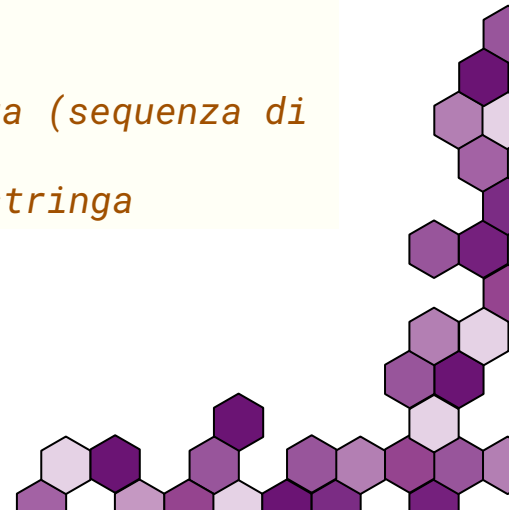
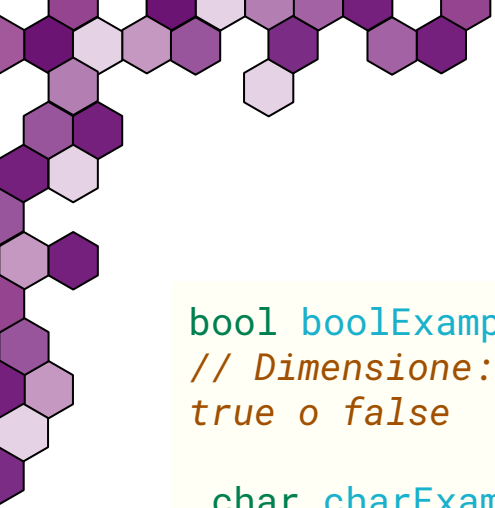


I tipi non numerici

```
bool boolExample = false; // Tipo booleano (vero o falso)  
// Dimensione: 1 byte (o più, dipende dall'implementazione), Valori:  
true o false
```

```
char charExample = 'A'; // Tipo carattere singolo Unicode (16-bit)  
// Dimensione: 2 byte, Valori: singoli caratteri Unicode (U+0000 a  
U+FFFF)
```

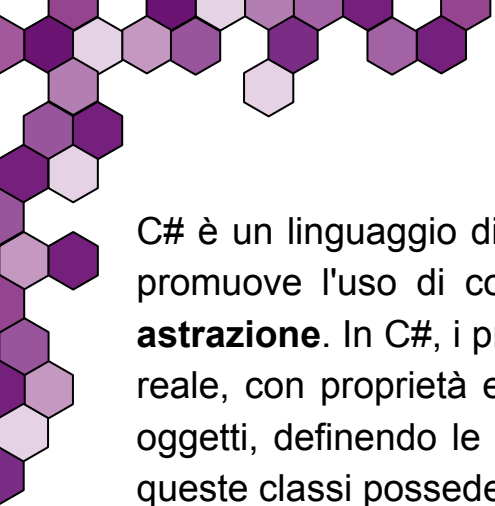
```
string stringExample = "Hello, World!"; // Tipo stringa (sequenza di  
caratteri Unicode)  
// Dimensione variabile, dipende dalla lunghezza della stringa
```



Il paradigma OOP

C# è un linguaggio di programmazione orientato agli oggetti (OOP), il che significa che supporta e promuove l'uso di concetti come **classi**, **oggetti**, **ereditarietà**, **composizione**, **polimorfismo** e **astrazione**. In C#, i programmi sono strutturati attorno a oggetti che rappresentano entità del mondo reale, con proprietà e comportamenti specifici. Le **classi** fungono da blueprint per la creazione di oggetti, definendo le caratteristiche (proprietà) e i comportamenti (metodi) che gli oggetti creati da queste classi possederanno.

In C#, anche i tipi di base, come `int`, `float`, `double`, e `bool`, sono in realtà derivati da una classe. Questi tipi, noti come **tipi primitivi** o **tipi valore**, sono supportati direttamente dal linguaggio per garantire efficienza e prestazioni. Tuttavia, in C#, ogni tipo di dato, inclusi i tipi di base, deriva implicitamente dalla classe `System.Object`, la classe base di tutti i tipi nel .NET Framework. Questo significa che, nonostante la loro natura primitiva, i tipi di base sono **oggetti** e possono utilizzare metodi e proprietà definite nella classe `System.Object`. Ad esempio, un `int` può essere convertito in una stringa usando il metodo `ToString()`, che è ereditato da `System.Object`.



La classe `System.Object`

La classe **`System.Object`** è la classe base di tutti i tipi in C# e nel .NET Framework, fornendo una struttura comune per tutti gli oggetti, sia per i tipi di riferimento che per i tipi di valore. Ogni tipo in C# eredita direttamente o indirettamente da `System.Object`, il che significa che tutti i tipi possono utilizzare i metodi definiti in questa classe.

Tra i metodi fondamentali forniti da `System.Object` ci sono:

- `ToString()`, che restituisce una rappresentazione stringa dell'oggetto
- `Equals(object obj)`, che determina se due oggetti sono uguali
- `GetHashCode()`, che restituisce un codice hash per l'oggetto
- `GetType()`, che restituisce il tipo dell'oggetto

Questo insieme di metodi fornisce funzionalità essenziali per la gestione e l'interazione con gli oggetti in C#, rendendo `System.Object` una parte cruciale del sistema di tipi del linguaggio. Quindi quando creiamo una nuova classe in C# sappiamo che questi tre metodi sono già presenti ma necessitano di eventuali modifiche.

La classe System.Object

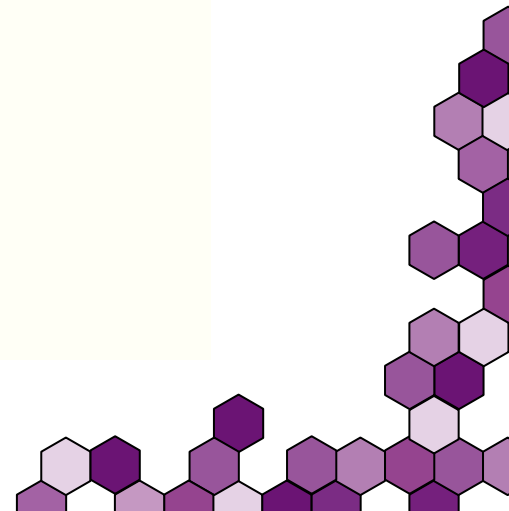
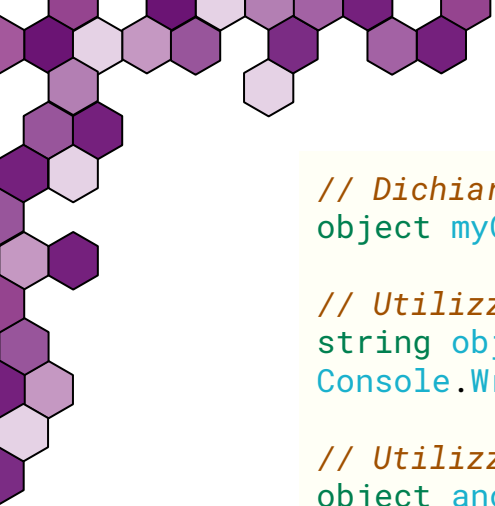
```
// Dichiarazione di un oggetto della classe System.Object
object myObject = new object();

// Utilizzo del metodo ToString()
string objectString = myObject.ToString();
Console.WriteLine(objectString); // Output: "System.Object"

// Utilizzo del metodo Equals()
object anotherObject = new object();
bool isEqual = myObject.Equals(anotherObject);
Console.WriteLine(isEqual); // Output: "False"

// Utilizzo del metodo GetHashCode()
int hashCode = myObject.GetHashCode();
Console.WriteLine(hashCode);

// Utilizzo del metodo GetType()
Type objectType = myObject.GetType();
Console.WriteLine(objectType); // Output: "System.Object"
```



La classe System.Object

```
// Creazione di un oggetto int
int number = 42;

// Utilizzo del metodo ToString() di System.Object
string numberString = number.ToString();
Console.WriteLine(numberString); // Output: "42"

// Utilizzo del metodo Equals() di System.Object
int anotherNumber = 42;
Console.WriteLine(number.Equals(anotherNumber)); // Output: "True"

// Utilizzo del metodo GetHashCode() di System.Object
int hashCode = number.GetHashCode();
Console.WriteLine(hashCode); // Output: codice hash dell'oggetto int 42

// Utilizzo del metodo GetType() per ottenere il tipo runtime dell'oggetto
Type typeOfNumber = number.GetType();
Console.WriteLine(typeOfNumber); // Output: int
```



Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



L'importanza di conoscere più opzioni

In informatica esistono molteplici approcci per risolvere lo stesso problema, ma è possibile cadere nella trappola di utilizzare sempre le stesse soluzioni. Ad esempio, si possono impiegare una serie di istruzioni **if** senza mai scrivere un **else**, oppure ricorrere a un solo tipo di ciclo. Tuttavia, questo non rappresenta il metodo ottimale per studiare l'informatica. Affidarsi costantemente alle stesse soluzioni può sembrare efficiente a breve termine, ma può diventare problematico a lungo termine. Un programmatore deve essere in grado di risolvere problemi in una vasta gamma di situazioni, scenari diversi e con strumenti simili ma distinti. La caratteristica più importante da sviluppare come informatici è, infatti, l'**elasticità**, ovvero la capacità di adattare il proprio approccio a ogni situazione che ci viene presentata.

L'importanza di conoscere più opzioni

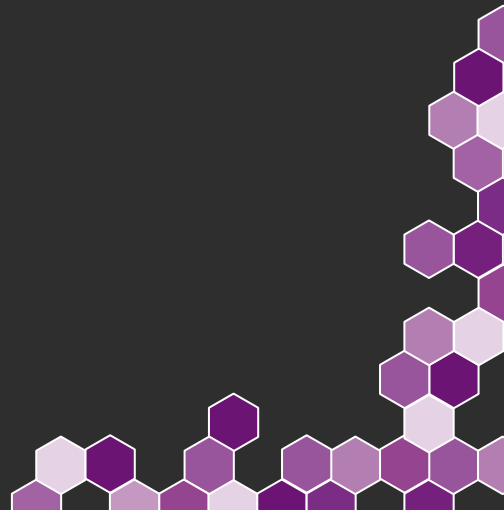
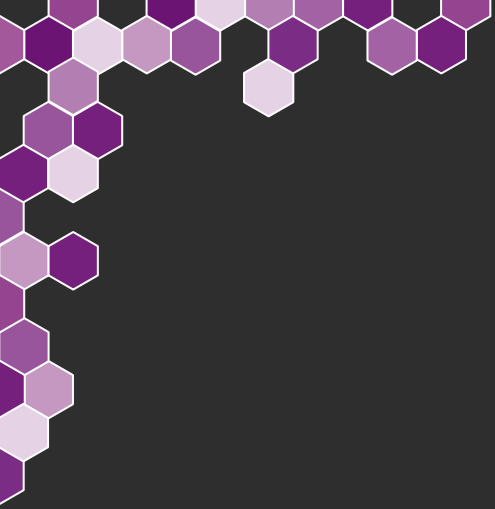
Donald Knuth, autore della monumentale opera "The Art of Computer Programming", ha sempre sottolineato l'importanza di comprendere a fondo gli algoritmi e le strutture dati per poter scegliere l'approccio giusto a seconda del contesto. Questo principio si riflette nel concetto di programmazione flessibile, dove l'**adattabilità** e la **capacità di scegliere** l'algoritmo più appropriato sono fondamentali per risolvere problemi complessi in modo efficiente.



Linus Torvalds, il creatore del kernel Linux, ha spesso parlato dell'importanza di mantenere una mente aperta e di non temere di esplorare nuove strade o di correggere errori. Torvalds stesso ha sostenuto che il successo di Linux è dovuto in parte alla comunità di sviluppatori che contribuiscono con diverse prospettive e **soluzioni innovative**. Questa **attitudine all'esplorazione** e all'adattamento ha permesso a Linux di evolversi costantemente, diventando uno dei sistemi operativi più robusti e versatili del mondo.



Condizioni



If Statement

```
if (condizione)
{
    // Codice da eseguire se la condizione è vera
}
```

```
if (condizione)
{
    // Codice da eseguire se la condizione è vera
}
else
{
    // Codice da eseguire se la condizione è falsa
}
```

If Statement

```
if (condizione1)
{
    // Codice da eseguire se la condizione1 è vera
}
else if (condizione2)
{
    // Codice da eseguire se la condizione2 è vera
}
else
{
    // Codice da eseguire se tutte le condizioni precedenti sono false
}
```

Switch Statement

```
switch (espressione)
{
    case valore1:
        // Codice da eseguire se espressione è uguale a valore1
        break;
    case valore2:
        // Codice da eseguire se espressione è uguale a valore2
        break;
    // Altri casi...
    default:
        // Codice da eseguire se nessun caso precedente è soddisfatto
        break;
}
```

Ternary Operator

```
var risultato = condizione ? valoreSeVero : valoreSeFalso;
```

```
int number = 10;  
string result;  
if (number > 0){  
    result = "Positive";  
}  
else {  
    result = "Non-positive";  
}  
Console.WriteLine(result); // Output: "Positive"
```

```
int number = 10;  
string result = (number > 0) ? "Positive" : "Non-positive";  
  
Console.WriteLine(result); // Output: "Positive"
```

Null-Coalescing Operator

```
var risultato = valore ?? valoreDefault;
```

```
Console.WriteLine("Per favore, inserisci il tuo nome :");  
string userInput = Console.ReadLine();
```

```
//Fornisce un valore predefinito se l'input è null o vuoto  
string name = userInput ?? "Ospite";  
Console.WriteLine($"Ciao, {name}!");
```

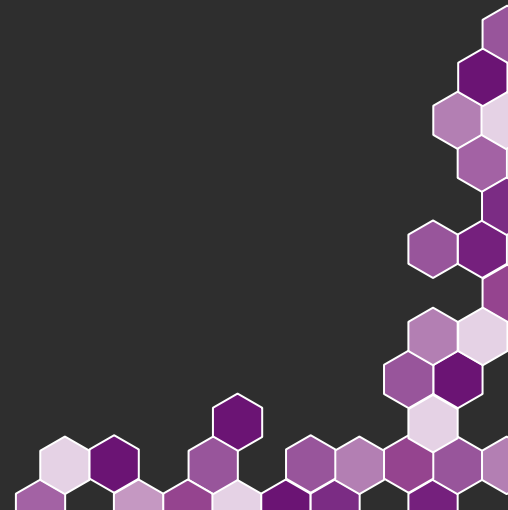
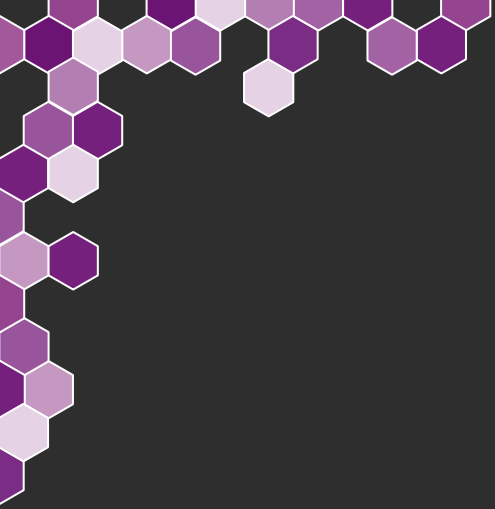


Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



Cicli



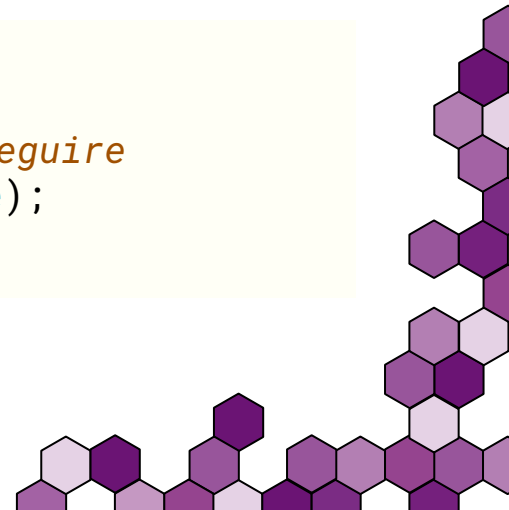
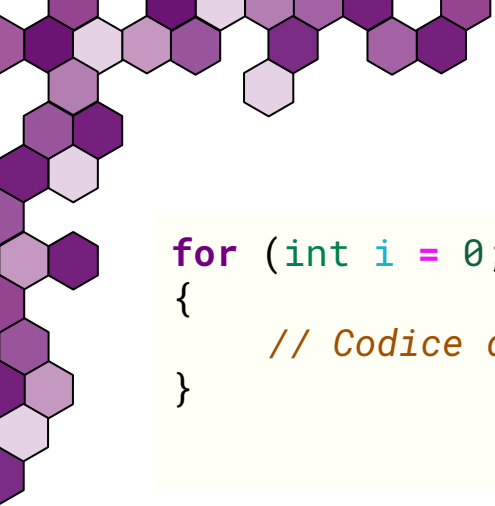
Loop

```
for (int i = 0; i < 10; i++)  
{  
    // Codice da eseguire  
}
```

```
foreach (var item in collezione)  
{  
    // Codice da eseguire  
}
```

```
while (condizione)  
{  
    // Codice da eseguire  
}
```

```
do  
{  
    // Codice da eseguire  
} while (condizione);
```



Loop

Descrizione:

- **Ciclo For:** Perfetto quando si conosce in anticipo il numero esatto di volte che un blocco di codice deve essere eseguito. Questo ciclo è strutturato con una dichiarazione iniziale, una condizione di continuazione e un incremento o decremento.
- **Ciclo While:** Utile quando il numero di iterazioni non è noto in anticipo e si basa su una condizione dinamica. È utile quando si desidera continuare l'esecuzione finché una condizione rimane vera.
- **Ciclo Do-While:** A differenza del ciclo while, il ciclo do-while verifica la condizione dopo aver eseguito il blocco di codice, garantendo che il codice venga eseguito almeno una volta, indipendentemente dalla condizione.
- **Ciclo Foreach:** Il ciclo foreach è specificamente progettato per iterare attraverso gli elementi di una collezione, come array, liste, o altre strutture che implementano IEnumerable. È ideale per scorrere ogni elemento senza dover gestire manualmente gli indici o le condizioni di fine ciclo.



Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



Array e matrici

Array

Un array è una struttura dati in C# che memorizza un numero fisso di elementi omogenei accessibili tramite indici.

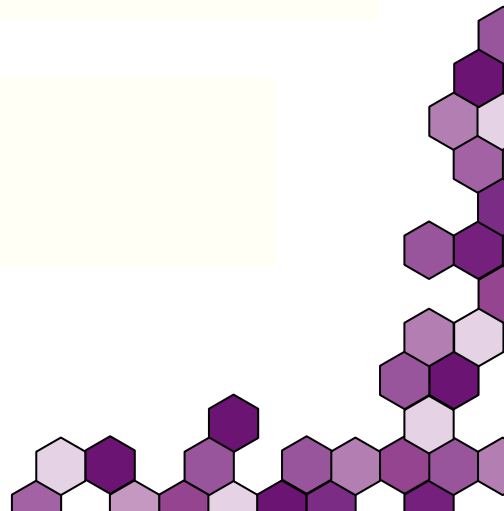
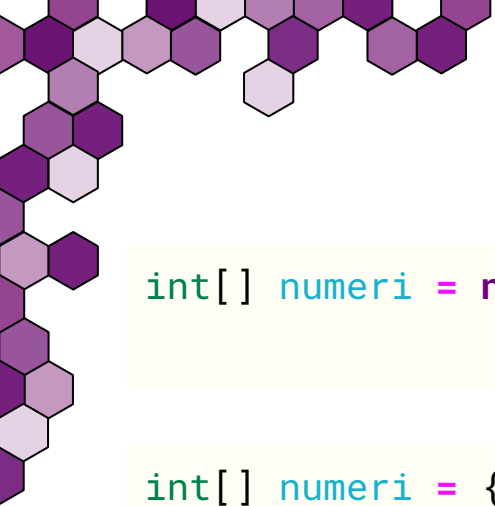
Gli array sono caratterizzati da una **omogeneità** dei dati, il che significa che tutti gli elementi contenuti devono essere dello stesso tipo, come interi, stringhe o qualsiasi altro tipo di dati definito. Un'altra caratteristica distintiva degli array è la loro **dimensione fissa**: una volta dichiarata la lunghezza dell'array, non può essere modificata. Questo implica che l'array non può crescere o ridursi dinamicamente durante l'esecuzione del programma. Gli elementi in un array sono **accessibili tramite un indice** numerico, che inizia da zero, permettendo l'accesso diretto e veloce agli elementi. Gli array sono ampiamente utilizzati per la gestione di collezioni di dati dove è noto il numero di elementi e si richiede una gestione ordinata ed efficiente degli stessi.

Array

```
int[] numeri = new int[5]; // Array di 5 interi
```

```
int[] numeri = { 5, 3, 8, 1, 4 }; // Inizializzazione diretta con valori
```

```
int primoNumero = numeri[0]; // Accesso al primo elemento  
numeri[2] = 10; // Modifica del terzo elemento
```

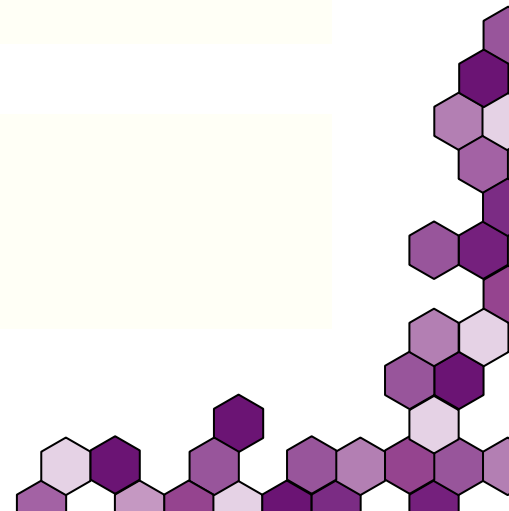
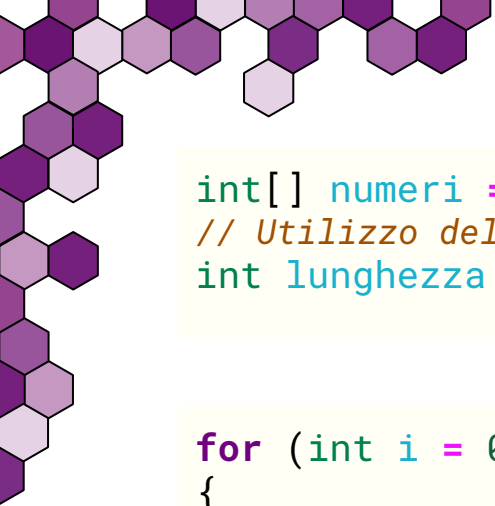


Array

```
int[] numeri = { 5, 3, 8, 1, 4 };  
// Utilizzo della proprietà Length per ottenere la lunghezza dell'array  
int lunghezza = numeri.Length;
```

```
for (int i = 0; i < numeri.Length; i++)  
{  
    Console.WriteLine(numeri[i]);  
}
```

```
foreach (int numero in numeri)  
{  
    Console.WriteLine(numero);  
}
```



Array

```
int[] numeri = { 5, 3, 8, 1, 4 };
```

```
Array.Sort(numeri);
```

// Utilizzo del metodo Array.Sort(array) per ordinare l'array

```
Console.WriteLine("Array ordinato:");
```

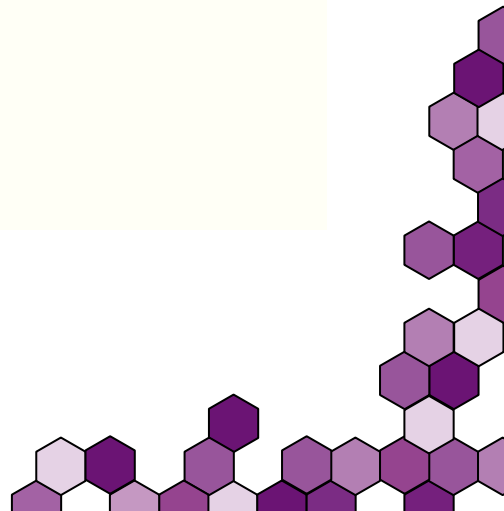
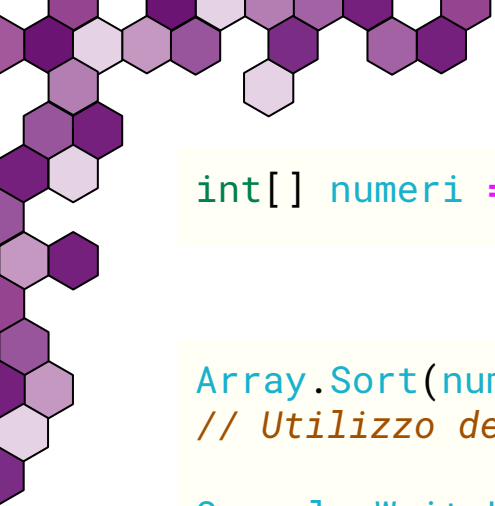
```
foreach (int numero in numeri)
```

```
{
```

```
    Console.Write(numero + " "); // Output: 1 3 4 5 8
```

```
}
```

```
Console.WriteLine();
```



Array

```
int[] numeri = { 5, 3, 8, 1, 4 };
```

```
Array.Reverse(numeri);
```

// Utilizzo del metodo Array.Reverse(array) per invertire l'ordine degli elementi dell'array

```
Console.WriteLine("Array invertito:");
```

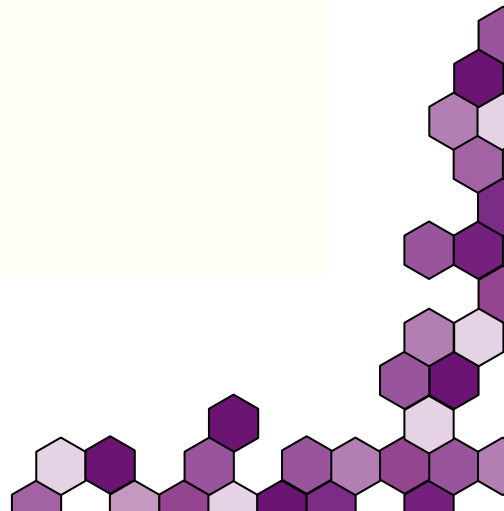
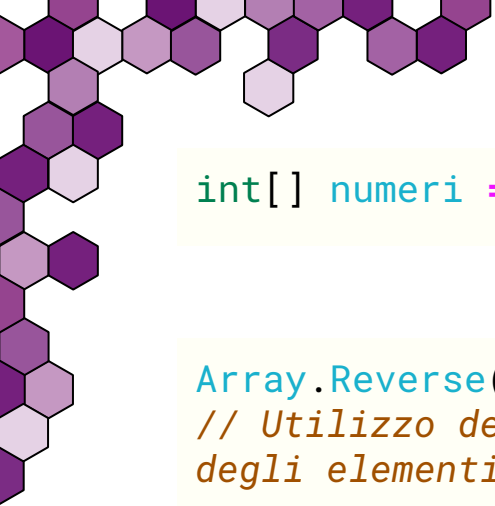
```
foreach (int numero in numeri)
```

```
{
```

```
    Console.Write(numero + " "); // Output: 8 5 4 3 1
```

```
}
```

```
Console.WriteLine();
```

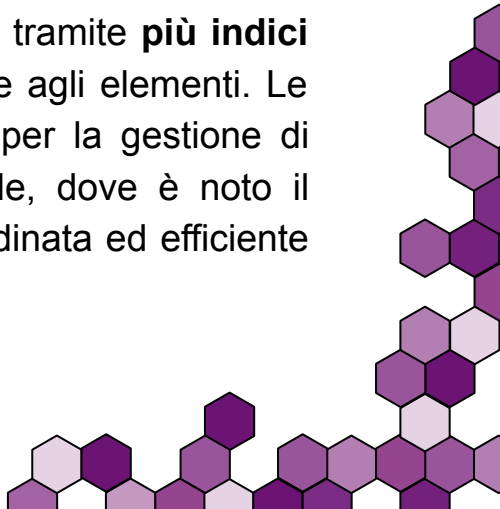




Matrici

Una matrice è una struttura dati in C# che memorizza elementi omogenei in una griglia multidimensionale, accessibili tramite più indici numerici.

Le matrici sono caratterizzate da una **omogeneità dei dati**, il che significa che tutti gli elementi contenuti devono essere dello stesso tipo, come interi, stringhe o qualsiasi altro tipo di dati definito. Un'altra caratteristica distintiva delle matrici è la loro **dimensione fissa**: una volta dichiarata la **lunghezza delle dimensioni**, queste non possono essere modificate. Questo implica che la matrice non può crescere o ridursi dinamicamente durante l'esecuzione del programma. Gli elementi in una matrice sono accessibili tramite **più indici numerici**, che iniziano da zero, permettendo un accesso diretto e veloce agli elementi. Le matrici sono ampiamente utilizzate per rappresentare tabelle di dati e per la gestione di collezioni di dati strutturati in forma bidimensionale o multidimensionale, dove è noto il numero di elementi in ciascuna dimensione e si richiede una gestione ordinata ed efficiente degli stessi.



Matrici

```
int[,] matrice = new int[3, 3]; // Matrice 3x3 di interi
```

```
int[,] matrice = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };  
// Matrice 3x3 di interi inizializzata
```

```
int elemento = matrice[0, 1];  
// Accesso all'elemento nella prima riga, seconda colonna  
matrice[2, 2] = 10;  
// Modifica dell'elemento nell'ultima riga e colonna
```

Matrici

```
int[, ] matrice = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

```
for (int i = 0; i < matrice.GetLength(0); i++)  
{  
    for (int j = 0; j < matrice.GetLength(1); j++)  
    {  
        Console.Write(matrice[i, j]); // Output: 1 2 3 4 5 6 7 8 9  
    }  
}
```

```
foreach (int valore in matrice)  
{  
    Console.Write(valore + " "); // Output: 1 2 3 4 5 6 7 8 9  
}
```

Matrici

```
int[, ] matrice = new int[3, 3];
```

```
int totalElements = matrice.Length; // Restituisce 9  
//Restituisce il numero totale di elementi presenti in tutte le  
dimensioni della matrice.
```

```
int dimensions = matrice.Rank; // Restituisce 2  
//Restituisce il numero di dimensioni della matrice.
```

```
int[, ] matrice = new int[3, 3];  
int rows = matrice.GetLength(0); //Restituisce 3 (numero di righe)  
int columns = matrice.GetLength(1); //Restituisce 3 (numero di colonne)  
//Restituisce il numero di elementi lungo una dimensione specificata.
```

Matrici

```
int[,] matrice = new int[3, 3];
```

```
int[,] matriceClone = (int[,])matrice.Clone();  
//Crea una copia superficiale della matrice.
```

```
int[] array = new int[9];  
matrice.CopyTo(array, 0);  
/*Copia tutti gli elementi della matrice in un array unidimensionale,  
a partire dall'indice specificato.*/*
```

Tuple

In C#, una **tupla** è uno strumento che permette di **raccogliere due o più valori eterogenei** in un'unica entità senza dover creare una struttura dati complessa. Nel caso più comune, una tupla può essere vista come un **coppia di valori** logicamente associati tra loro, ad esempio **nome e voto**, oppure **codice e descrizione**.

Le tuple sono **tipizzate**, **ordinate** e **immutabili**: questo significa che ogni valore ha un tipo definito, mantiene la sua posizione (Item1, Item2, ecc.), e non può essere modificato dopo la creazione, se non tramite riassegnazione.

```
(double, int) t1 = (4.5, 3);  
Console.WriteLine($"Tuple with elements {t1.Item1} and {t1.Item2}.");  
// Output:  
// Tuple with elements 4.5 and 3.
```

```
(double Sum, int Count) t2 = (4.5, 3);  
Console.WriteLine($"Sum of {t2.Count} elements is {t2.Sum}.");  
// Output:  
// Sum of 3 elements is 4.5.
```

Dizionari

Un dizionario è una struttura dati in C# che memorizza coppie chiave-valore, consentendo l'accesso ai valori tramite chiavi uniche.

I dizionari sono caratterizzati dalla capacità di associare **chiavi uniche** a valori, dove ogni chiave è unica e viene utilizzata per identificare il valore corrispondente. Questo rende i dizionari estremamente efficienti per operazioni di ricerca, inserimento e rimozione, poiché ogni operazione ha una complessità temporale media costante, $O(1)$. Un'altra caratteristica distintiva dei dizionari è la loro **flessibilità**: a differenza delle matrici o degli array, la dimensione di un dizionario può crescere dinamicamente durante l'esecuzione del programma man mano che nuovi elementi vengono aggiunti. Gli elementi in un dizionario sono accessibili tramite la loro chiave, permettendo un rapido recupero del valore associato. I dizionari sono ampiamente utilizzati per la gestione di collezioni di dati dove è importante l'associazione rapida e efficiente tra chiavi e valori.

Dizionari

```
Dictionary<TKey, TVal> dizGenerico = new Dictionary<TKey, TVal>();
```

```
Dictionary<int, string> dizionario = new Dictionary<int, string>();
```

```
Dictionary<string, int> dizStringInt = new Dictionary<string,  
int>();
```

```
Dictionary<int, int[]> dizIntArray = new Dictionary<int, int[]>();
```

Dizionari

```
// Dichiarazione e inizializzazione di un dizionario  
Dictionary<int, string> dizionario = new Dictionary<int, string>();
```

```
// Aggiunta di elementi al dizionario  
dizionario.Add(1, "Uno");  
dizionario.Add(2, "Due");  
dizionario.Add(3, "Tre");
```

```
// Accesso ai valori tramite chiavi  
Console.WriteLine("Il valore della chiave 1 è: " + dizionario[1]);  
// Output: Uno  
Console.WriteLine("Il valore della chiave 2 è: " + dizionario[2]);  
// Output: Due
```

Dizionari

```
// Dizionario con chiavi di tipo string e valori di tipo int  
Dictionary<string, int> dizStringInt = new Dictionary<string, int>();
```

```
// Aggiunta di elementi al dizionario  
dizStringInt.Add("Uno", 1);  
dizStringInt.Add("Due", 2);
```

```
// Accesso a un valore tramite la chiave  
int valoreUno = dizStringInt["Uno"]; // 1
```

```
// Iterazione su tutti gli elementi del dizionario  
foreach (KeyValuePair<string, int> kvp in dizStringInt)  
{  
    Console.WriteLine("Chiave: " + kvp.Key + ", Valore: " + kvp.Value);  
}
```

Dizionari

```
// Dizionario con chiavi di tipo int e valori di tipo int[]  
Dictionary<int, int[]> dizIntArray= new Dictionary<int, int[]>();
```

```
// Aggiunta di elementi al dizionario  
dizIntArray.Add(1, new int[] { 1, 2, 3 });  
dizIntArray.Add(2, new int[] { 4, 5, 6 });
```

```
// Accesso a un array tramite la chiave  
int[] array1 = dizIntArray[1]; // { 1, 2, 3 }  
  
int val1 = dizIntArray[1][0]; // 1
```

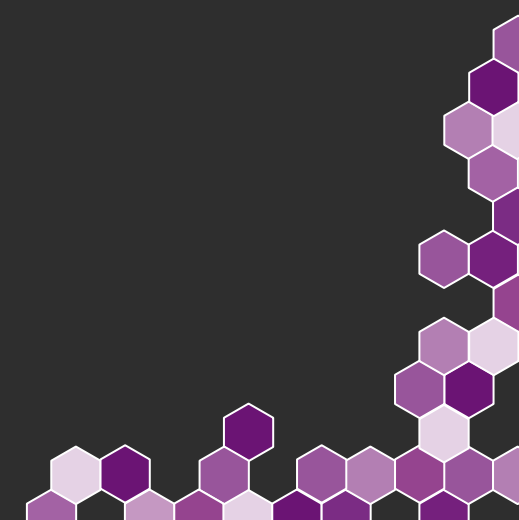
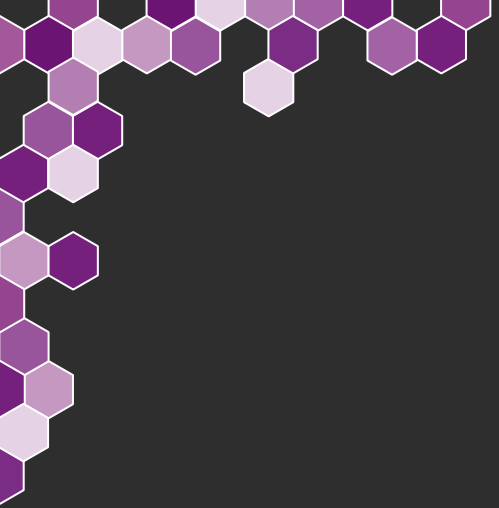


Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni



Funzioni



Funzioni

Una funzione in C# è composta da diversi elementi distinti che insieme definiscono la sua struttura.

La dichiarazione di una funzione inizia con un **modificatore di accesso**, come **public** e **private** che determina la visibilità della funzione all'interno e all'esterno della classe in cui è definita.

Segue il **tipo di ritorno**, che specifica il tipo di dato che la funzione restituirà dopo l'esecuzione; se la funzione non restituisce alcun valore, si utilizza la parola chiave **void**.

Successivamente, viene indicato il **nome della funzione**, che dovrebbe essere descrittivo del compito che svolge.

Dopo il nome, tra parentesi tonde, si trova la **lista dei parametri**, che specifica i dati di input che la funzione può ricevere.

Infine, il **corpo della funzione**, racchiuso tra parentesi graffe, contiene il blocco di codice che esegue l'operazione desiderata.

Funzioni

```
public int Somma(int a, int b)
{
    return a + b;
}
```

```
public void StampaMessaggio()
{
    Console.WriteLine("Ciao, mondo!");
}
```

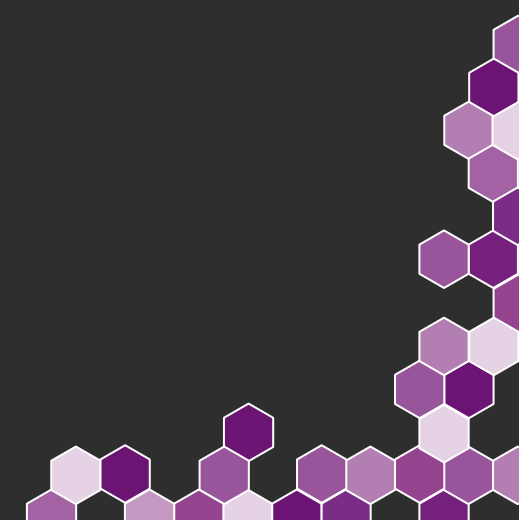
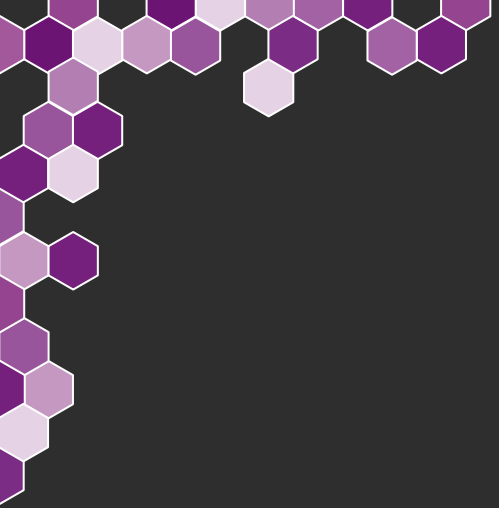



Indice:

- Visual Studio, l'IDE della microsoft
- App console presentazione e IO
- Tipi di variabili
- Condizioni
- Cicli
- Strutture dati semplici
- Funzioni
- Esercizio di esempio svolto



Esercizio



Gestione di Prenotazioni per un Cinema

In qualità di custode del cinema Paradiso che vanta una lunga tradizione, è essenziale disporre di un sistema che ci consenta di gestire le prenotazioni in modo ordinato e puntuale. A tal fine, proponiamo lo sviluppo di un'applicazione console dedicata alla gestione delle prenotazioni dei nostri preziosi spettatori. Questa applicazione dovrebbe offrire le seguenti funzionalità:

1. Visualizzare la disposizione dei posti di una sala da 64 posti e controllare la disponibilità di posti mostrando liberi e prenotati.
2. Prenotare un posto specificando il numero di riga e colonna.
3. Annullare una prenotazione.
4. Calcolare il totale delle prenotazioni effettuate.

Ringraziando per la disponibilità:

Giulio Rossi



Analisi del test

Gestione di Prenotazioni per un Cinema

In qualità di custode del **cinema Paradiso** che vanta una lunga tradizione, è essenziale disporre di un sistema che ci consenta di gestire le prenotazioni in modo **ordinato** e puntuale. A tal fine, proponiamo lo sviluppo di un'**applicazione console** dedicata alla gestione delle prenotazioni dei nostri preziosi spettatori. Questa applicazione dovrebbe offrire le seguenti funzionalità:

1. **Visualizzare la disposizione dei posti** di una sala da **64 posti** (Rappresentare come 8x8?) e controllare la disponibilità di posti mostrando liberi e prenotati. (Da fare assieme? Stampare l'intera sala magari segnalando cosa è libero e cosa no?)
2. **Prenotare un posto** specificando il numero di **riga e colonna**.
3. **Annullare una prenotazione**. (magari da mettere prenotazione e annullamento vicine)
4. **Calcolare il totale delle prenotazioni effettuate**. (Da effettuare come chiusura?)

Noto che deve esserci una selezione iniziale dove possiamo selezionare cosa vogliamo fare, poi serve una serie di funzioni che si occupino di svolgere le singole operazioni. Ci sono delle decisioni da prendere e per semplicità diciamo sì a tutte le domande che mi sono poste.

Struttura del programma

Descrizione delle Funzioni:

1. **Main**: Gestisce il menu principale e le opzioni selezionate dall'utente.
2. **VisualizzaDisponibilita**: Conta e visualizza il numero di posti liberi e occupati.
3. **PrenotaPosto**: Permette all'utente di prenotare un posto, se disponibile.
4. **AnnullaPrenotazione**: Annulla una prenotazione esistente.
5. **Chiusura**: Funzione per calcolare il totale delle prenotazioni effettuate e reimpostare il programma allo stato di partenza

Mi avvalgo della **possibilità di creare sotto funzioni** per delegare sotto parti del programma, il passaggio successivo è impostare le **firme di queste funzioni**. Inoltre serve **definire la matrice** come variabile **globale** e sarà compito del main inizializzarla. Inoltre scelgo che la matrice sarà di **booleani**.

Firme delle funzioni

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CinemaParadiso
{
    static bool[,] sala = new bool[8, 8];
    //Unico posto dove viene esplicitata la dimensione
    internal class Program
    {
        static void Main(string[] args) { }
        static void VisualizzaDisponibilita() { }
        static void PrenotaPosto() { }
        static void AnnullaPrenotazione() { }
        static void Chiusura() { }
    }
}
```

Main

```
//Inizializzazione della matrice sala
for (int i = 0; i < sala.GetLength(0); i++)
{
    for (int j = 0; j < sala.GetLength(1); j++)
    {
        sala[i, j] = false;
        //decido che false è posto libero mentre true posto occupato
    }
}
```

In questa implementazione, il valore **false** indica un **posto libero**, mentre **true** rappresenta un **posto occupato**. Questa scelta è giustificata dalla natura binaria degli stati di un posto, che può essere o disponibile o prenotato. Visto che il programma gestisce le prenotazioni secondo me è più logico vedere il posto prenotato come **true** perché è vero che c'è una prenotazione. Ma qualcun altro potrebbe vedere **true** come il posto libero e quindi invertire il paradigma.

Main

```
// Variabile booleana che controlla il ciclo del menu principale che serve per far uscire.
bool uscita = false;

while (!uscita)
{
    // Pulisce la console, rimuovendo tutte le righe precedentemente stampate.
    Console.Clear();
    // Stampa il menu delle opzioni disponibili per l'utente.
    Console.Write("Benvenuti al Cinema Paradiso!");
    Console.WriteLine("Scegli quale opzione più si avvicina alle tue necessità");
    Console.WriteLine("1. Visualizza disponibilità posti");
    Console.WriteLine("2. Prenota un posto");
    Console.WriteLine("3. Annulla una prenotazione");
    Console.WriteLine("e. Esci");
    Console.Write("Seleziona un'opzione: ");

    string scelta = Console.ReadLine(); // Legge l'input dell'utente e lo memorizza nella
                                         //variabile 'scelta' sarà confrontata nelle successive
                                         //istruzioni switch.
//Attenzione che il WHILE continua a slide dopo
}
```


Main

```
switch (scelta)
{
    case "1":
        // Esegue VisualizzaDisponibilita che conta e visualizza il numero di posti liberi e occupati.
        VisualizzaDisponibilita();

        break;
    case "2":
        // Esegue PrenotaPosto per permette all'utente di prenotare un posto, se disponibile.
        PrenotaPosto();

        break;
    case "3":
        // Esegue e AnnullaPrenotazione per annullare una prenotazione esistente.
        AnnullaPrenotazione();
        break;
    case "e":
        uscita = true; // Termina così il ciclo while e quindi l'esecuzione del programma.
        // Inoltre, viene chiamata Chiusura che mostra le prenotazioni totali e resetta il programma.
        Chiusura();
        Console.ReadKey();
        break;
    default:
        // Se l'utente inserisce un'opzione non valida, viene visualizzato un messaggio di errore
        Console.WriteLine("Scelta non valida. Premere un tasto per continuare...");
        Console.ReadKey();
        break;
}
} // Chiusura del WHILE della slide precedente
```

VisualizzaDisponibilita

```
static void VisualizzaDisponibilita()
{
    Console.Clear(); // Pulisce la console per presentare un'interfaccia utente pulita
    Console.WriteLine("Disposizione dei posti (0 = libero, X = occupato):");
    // Visualizzazione dei posti, '0' rappresenta un posto libero e 'X' un posto occupato.
    for (int i = 0; i < sala.GetLength(0); i++)
    {
        Console.Write("[ ");
        // Inizia la riga della matrice dei posti.
        for (int j = 0; j < sala.GetLength(1); j++)
        {
            // Controlla se il posto in posizione [i,j] è occupato (true) o libero (false).
            // Se occupato quindi true, stampa "X", altrimenti stampa "0".
            if (sala[i,j])
                Console.Write("X ");
            else
                Console.Write("0 ");
        }
        Console.WriteLine("]");
        // Chiude la riga corrente della matrice e passa alla riga successiva.
    }
    Console.WriteLine("Premere un tasto per continuare...");
    Console.ReadKey();
}
```

VisualizzaDisponibilita2

```
static void VisDisp2()
{
    Console.Clear();
    // Stampa delle lettere per le colonne
    Console.Write(" "); // Spazi iniziali per allineare le lettere con i numeri delle righe
    for (int j = 0; j < sala.GetLength(1); j++)
    {
        Console.Write((char)('A' + j) + " "); // Colonne identificate con le lettere spostate con ASCII
    }
    Console.WriteLine();

    // Stampa della matrice con numeri per le righe
    for (int i = 0; i < sala.GetLength(0); i++)
    {
        // Stampa del numero della riga
        Console.Write((i + 1) + " [ ");
        for (int j = 0; j < sala.GetLength(1); j++)
        {
            if (sala[i, j])
                Console.Write("X ");
            else
                Console.Write("0 ");
        }
        Console.WriteLine("]");
    }
    Console.WriteLine("Premere un tasto per continuare...");
    Console.ReadKey();
}
```

PrenotaPosto

```
static void PrenotaPosto()
{
    Console.Clear();
    Console.WriteLine("Benvenuto nella prenotazione:");
    // Chiamate a due funzioni, PosRiga e PosColonna, per ottenere rispettivamente gli indici
    int colonna = PosColonna(sala.GetLength(0));
    int riga = PosRiga(sala.GetLength(1));
    //Mi sono avvalso della possibilità di usare sotto funzioni

    if (!sala[riga, colonna])
    {
        // Se il posto è libero (false), lo stato del posto viene cambiato a occupato (true).
        sala[riga, colonna] = true;
        Console.WriteLine("Prenotazione effettuata con successo.");
    }
    else
    {
        // Se il posto è occupato (true), viene notificato che il posto non è disponibile.
        Console.WriteLine("Il posto è già occupato.");
    }
    Console.WriteLine("Premere un tasto per continuare...");
    Console.ReadKey();
}
```

PosColonna

```
static int PosColonna(int len)
{
    int colonna = -1; // Valore iniziale invalido per controllare l'input
    bool inputValido = false;
    char input = ' ';
    while (!inputValido)
    {
        Console.WriteLine("Inserisci la lettera della colonna (A-"+ (char)('A' + len - 1) + "): ");
        // Richiede all'utente di inserire il numero della colonna (compreso tra A e len in ASCII)
        input = Console.ReadLine().ToUpper()[0]; // Legge l'input dell'utente e lo converte in maiuscolo

        if (input >= 'A' && input < 'A' + len )
        {
            colonna = input - 'A'; // Converte la lettera in un indice di colonna (A = 0, B = 1, ...)
            inputValido = true;
        }
        else
        {
            Console.WriteLine("Input non valido. Riprova.");
        }
    }

    return colonna;
}
```

PosRiga

```
static int PosRiga(int len)
{
    int riga = -1; // Valore iniziale invalido per controllare l'input
    int input = '';

    while (riga < 0)
    {
        Console.WriteLine("Inserisci il numero della riga (1-" + len.ToString() + "): ");
        // Richiede all'utente di inserire il numero della riga (compreso tra 1 e len)
        input = Console.ReadLine()[0] - 48;
        // Legge il primo carattere dell'input e lo converte in un intero ('0' ha valore ASCII di 48).
        if ( input >= 1 && input <= len)
        {
            riga = input - 1; // Converte il numero in un indice di riga (1 = 0, 2 = 1, ..., 8 = 7)
        }
        else
        {
            Console.WriteLine("Input non valido. Riprova.");
        }
    }

    return riga;
}
```

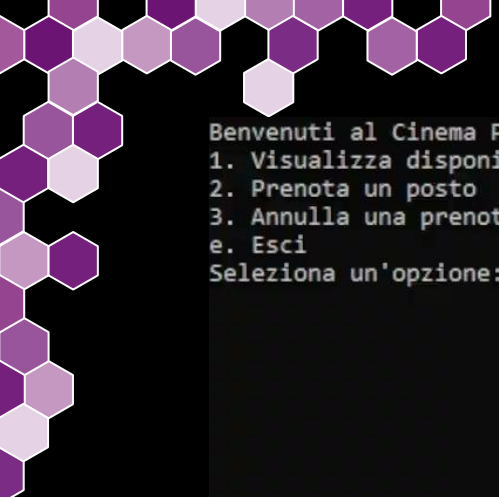
AnnullaPrenotazione

```
static void AnnullaPrenotazione()
{
    Console.Clear();
    Console.WriteLine("Siamo molto rammaricati che hai deciso di annullare la tua prenotazione");
    // Chiamate a due funzioni, PosRiga e PosColonna, per ottenere rispettivamente gli indici
    int colonna = PosColonna(sala.GetLength(0));
    int riga = PosRiga(sala.GetLength(1));

    if (sala[riga, colonna])
    {
        // Se il posto è occupato (true), lo stato del posto viene cambiato a .
        sala[riga, colonna] = false;
        Console.WriteLine("Hia annullato la tua prenotazione con successo.");
    }
    else
    {
        // Se il posto è libero (false), viene notificato che il posto è già disponibile..
        Console.WriteLine("Il posto è già libero.");
    }
    Console.WriteLine("Premere un tasto per continuare...");
    Console.ReadKey();
}
```

Chiusura

```
static void Chiusura()
{
    int totalePrenotazioni = 0;
    // Iterare su tutte le righe e colonne della matrice sala
    foreach (bool posto in sala)
    {
        if (posto)
        {
            totalePrenotazioni++;
        }
        // Stampare il totale delle prenotazioni effettuate
    }
    Console.WriteLine("Totale prenotazioni effettuate: " +
        totalePrenotazioni);
}
```

Benvenuti al Cinema Paradiso! Scegli quale opzione più si avvicina alle tue necessità

1. Visualizza disponibilità posti
2. Prenota un posto
3. Annulla una prenotazione
- e. Esci

Seleziona un'opzione:

Sistema di Gestione degli Ordini di un Ristorante

Ciao a tutti, sono Marco Conti, il proprietario della Trattoria da Marco, un luogo dove ogni cliente è trattato come un amico di lunga data. Per migliorare l'efficienza del nostro servizio e continuare a offrire un'esperienza calorosa e accogliente, desidero un'applicazione console per gestire gli ordini dei clienti. L'applicazione deve consentire di:

1. Visualizzare il menu con una serie di piatti e i relativi prezzi.
2. Effettuare un ordine selezionando piatti dal menu e specificando le quantità.
3. Calcolare e visualizzare il totale dell'ordine.
4. Applicare uno sconto del 5% se l'importo totale supera una certa soglia.
5. Visualizzare il riepilogo dell'ordine e l'importo finale.



Sistema di Gestione degli Ordini di un Ristorante

Ciao a tutti, sono Marco Conti, il proprietario della Trattoria da Marco, un luogo dove ogni cliente è trattato come un amico di lunga data. Per migliorare l'efficienza del nostro servizio e continuare a offrire un'esperienza calorosa e accogliente, desidero un'applicazione form strutturata a classi, con gestione delle eccezioni e salvataggio dei dati in file CSV, per gestire gli ordini dei clienti.

Requisiti dell'applicazione

1. Classi principali

- Piatto: rappresenta un piatto del menu (nome, prezzo).
- Menu: contiene una lista di piatti e i metodi per visualizzarli e cercarli.
- Ordine: rappresenta l'insieme dei piatti ordinati da un cliente con le relative quantità e l'identificativo cliente
- Ristorante: gestisce l'intero flusso (caricamento del menu da file CSV, gestione ordini, calcolo totale, applicazione sconti, salvataggio degli ordini, emissione scontrino).

2. Eccezioni personalizzate

- PiattoNonTrovatoException: lanciata se il cliente inserisce un codice o un nome di piatto non presente nel menu.
- QuantitaNonValidaException: lanciata se la quantità specificata non è positiva.
- FileMenuNonTrovatoException: lanciata se il file CSV del menu non è disponibile o corrotto.

3. Gestione file CSV

- Lettura del menu da un file menu.csv, dove ogni riga contiene NomePiatto;Prezzo.
- Salvataggio degli ordini su ordini.csv, con righe del tipo Cliente;Piatto;Quantità;PrezzoTotale.

4. Funzionalità richieste

- Visualizzare il menu con piatti e relativi prezzi.
- Effettuare un ordine selezionando i piatti dal menu e specificando le quantità.
- Gestire eventuali inserimenti errati (piatti inesistenti o quantità non valide).
- Calcolare e visualizzare il totale dell'ordine.
- Applicare uno sconto del 5% se l'importo totale supera una certa soglia (ad esempio 50 €).
- Visualizzare il riepilogo dell'ordine e l'importo finale.
- Salvare l'ordine in un file ordini.csv per tenere traccia della cronologia.

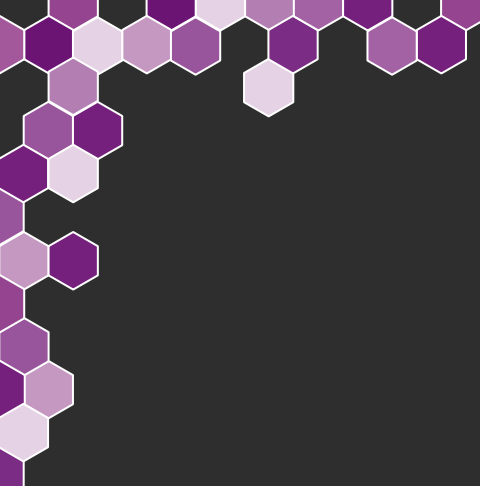
Gestione di Inventario per un Negozio di Elettronica

Salve, sono Raj Patel, il proprietario di **TechSavvy**, il miglior negozio di elettronica dove la tecnologia incontra l'innovazione. Abbiamo bisogno di un'applicazione console per gestire l'inventario dei nostri prodotti. L'applicazione deve consentire di:

1. Visualizzare l'inventario attuale, inclusi il nome del prodotto, la quantità e il prezzo unitario.
2. Aggiungere un nuovo prodotto all'inventario.
3. Aggiornare la quantità di un prodotto esistente.
4. Rimuovere un prodotto dall'inventario.
5. Calcolare il valore totale dell'inventario.

Grazie per aiutarci a mantenere TechSavvy all'avanguardia nella gestione dell'inventario!





Buono studio